

FORKROUTE

Food Delivery Marketplace Platform Modernization

Enterprise Architecture Definition Document

A TOGAF Architecture Development Method (ADM) Case Study

Preliminary Phase through Phase H — Full ADM Cycle

Prepared by: Enterprise Architecture Function

Document Reference: FRT-EA-FDM-001 | Version 1.0

August 2026

FICTIONAL CASE STUDY — ForkRoute, all named individuals, and all figures in this document are invented for demonstration and portfolio purposes. This document does not describe any real food-delivery company, and no figures herein should be relied upon as real-world data.

Table of Contents

1. Executive Summary.....	4
2. Preliminary Phase — Governance & Principles.....	5
2.1 Architecture Principles.....	5
2.2 Governance Framework Setup.....	6
3. Phase A — Architecture Vision & Scope.....	8
3.1 Problem Statement & Target-State Vision.....	8
3.2 Scope Boundaries.....	8
3.3 Success Metrics.....	9
3.4 Stakeholder Map.....	9
3.5 Business Case Summary.....	10
4. Phase B — Business Architecture.....	12
4.1 As-Is Business Architecture.....	12
4.2 To-Be Business Architecture.....	12
4.3 Business Capability Map.....	13
4.4 Business Footprint.....	14
5. Phase C — Information Systems Architecture.....	16
5.1 Application Architecture.....	16
5.2 Data Architecture.....	17
5.3 Data Ownership / Domains.....	18
6. Phase D — Technology Architecture.....	20
6.1 Reference Architecture Pattern.....	20
6.2 Technology Standards.....	20
7. Phase E — Opportunities & Solutions.....	22
7.1 Gap Analysis.....	22
7.2 Solution Building Blocks.....	22
7.3 Vendor Evaluation.....	23
8. Phase F — Migration Planning.....	25
8.1 Migration Roadmap.....	25
8.2 Transition Architecture — Home Market Dual-Run.....	25
9. Phase G — Implementation Governance.....	27
9.1 Non-Compliance Handling.....	27
9.2 Architecture Contracts.....	27
10. Phase H — Change Management.....	28

11. Architecture Decision Records.....	30
Appendix A — Diagram Catalog Index.....	31
Appendix B — Disclaimer.....	32

1. Executive Summary

ForkRoute operates a food delivery marketplace serving roughly 2 million monthly active users, 15,000 restaurant partners, and 40,000 active riders in a single home market. Leadership has committed to launching in four additional countries within 18 months, and three structural properties of the current platform stand directly in the way. Order-dispatch and driver-allocation logic is hardcoded per city — hand-coded, drifting, with no authoritative source of truth and an 8–10 week lead time to encode a new market's rules. The restaurant partner integration platform is a shared-fate monolith with no per-tenant isolation, so a single partner's incident can degrade order intake platform-wide — this has happened, on the current 15,000-partner base, an estimated 6–8 times per year. And payments reconciliation across customers, restaurants, and riders runs as nightly batch, which cannot support the real-time payouts or dynamic surge pricing the expansion markets are being positioned to require competitively.

The Enterprise Architecture function recommends re-architecting three core systems — dispatch, payments/reconciliation, and restaurant integration — around a configuration-driven, event-based platform that treats "add a market" as an authoring exercise, not an engineering project. A config-driven dispatch engine (ADR-001) loads market-specific rule bundles at runtime; an event-driven payments backbone (ADR-002) maintains payment and payout state as an append-only, replayable log; a multi-tenant restaurant integration platform (ADR-003) isolates each partner's blast radius; and data residency (ADR-005) is designed in as a jurisdiction-aware partitioning dimension rather than retrofitted market by market. This is a re-architecture, not a rebuild — the home market continues operating throughout, migrating in stages through a named transition architecture (Transition State Alpha) before any market cuts over.

Total program investment is estimated at \$4,696,600 over 18 months, including a 15% contingency on a \$4,084,000 capex subtotal. Per-market rollout cost drops from an estimated \$420,000 to \$110,000, saving roughly \$1,240,000 in direct rollout cost across the four target markets. Steady-state annual opex rises only marginally — a net +\$25,000/year — as higher infrastructure and vendor-licensing cost is nearly offset by lower incident and maintenance cost. Read honestly, the 3-year TCO is higher for the re-architected platform (\$16,196,600) than for continuing to expand on the current one (\$12,690,000): this is disclosed transparently rather than obscured, because the business case is not primarily a cost-reduction case. Payback on cost savings alone is not achieved within 3 years — the program is justified instead by avoided cost of delay (an illustrative \$2.1M per month of deferred contribution margin, meaning the program's capex is recovered by avoiding roughly 2.2 months of expansion delay) and by risk avoidance: the current integration platform's incident pattern, extrapolated to 2.5x partner count with no isolation, carries a materially higher probability of a platform-wide outage in a new market where ForkRoute has no brand-trust cushion.

The migration runs across a Foundation period and four waves over the full 18-month program: Foundation (Months 1–5) builds the data residency zone architecture, payments backbone, and dispatch engine against home-market production traffic without cutting over; Wave 1 (Months 6–8) dual-runs the new platform against the legacy home-market systems before full cutover; Waves 2 through 4 (Months 9–18) launch Market A, then Markets B and C in parallel, then Market D, closing with a formal governance handover from the program's Architecture Review Board to ForkRoute's standing engineering organization. This document is the consolidated Architecture Definition Document for the program, compiled across every phase of the TOGAF Architecture Development Method — Preliminary Phase governance setup through Phase H change management — and is intended as a single, self-contained reference of the architecture, its rationale, and its governance.

2. Preliminary Phase — Governance & Principles

Prior to this program, ForkRoute had no standing architecture review function — city-level engineering leads made local technology and design decisions independently, which is the direct cause of the per-city dispatch-logic drift described in Phase A. An expansion into four new markets in 18 months cannot tolerate that pattern repeating at a larger scale, since the cost of drift multiplies with each market added under the old model. Formal governance is therefore a precondition for the expansion business case, not an optional overlay, and was stood up in the Preliminary Phase before any Phase A vision work began.

2.1 Architecture Principles

Ten architecture principles were ratified by the Architecture Review Board (ARB) and govern every subsequent decision in this program. Each principle follows the standard TOGAF format — Statement, Rationale, Implications — and each is enforceable by the ARB; every ADR and architecture contract in this document cites these principles by number. The numbering is not a priority ranking.

#	Principle	Statement (Condensed)
1	Configuration Over Code for Market Variation	Market-specific business rules (dispatch radius, surge thresholds, payout timing, tax treatment, currency) are expressed as versioned configuration evaluated at runtime, not as code branches or per-market service forks.
2	Tenant Isolation Is Non-Negotiable at the Integration Boundary	Any system integrating with an external party (restaurant partner, payment processor, logistics provider) must contain that party's failure or traffic anomaly to that party alone.
3	Payments State Is Event-Sourced, Not Batch-Reconciled	The authoritative record of a payment's state is an append-only event log; nightly batch reconciliation is a downstream audit function, not the system of record.
4	Data Residency Follows the Market, Not the Platform	Personally identifiable customer, rider, and restaurant data is stored and processed within the legal jurisdiction the data subject operates in, by design, not as a post-hoc exception.
5	Every New Market Is a Configuration, Not a Fork	Onboarding a new country or city is achieved by authoring a new configuration bundle against the existing platform, never by branching or duplicating a service.
6	Interfaces Are Contracts, Versioned and Backward-Compatible	All service-to-service and partner-facing interfaces are explicit, versioned contracts; breaking changes require a new version published alongside the old, with a deprecation window.
7	Build Only What Differentiates ForkRoute	Undifferentiated capability (route optimization, payment gateway connectivity, KYC/AML checks) is bought or integrated; engineering effort is reserved for dispatch logic, marketplace dynamics, and partner experience.
8	Architecture Decisions Are Recorded, Not Tribal	Any decision that is expensive to reverse, affects more than one team, or trades off a named alternative is captured as a written Architecture Decision Record before implementation begins.
9	Design for Multi-Currency and Multi-Language From Day One	All customer-, rider-, and restaurant-facing systems and all financial calculations treat currency and locale as required parameters, never as an assumed constant.

#	Principle	Statement (Condensed)
10	Resilience Is Scoped to Business Impact, Not Uniformly Maximized	Availability and disaster-recovery targets are set per capability based on the cost of an outage to the business, not applied uniformly at the highest tier across the platform.

2.2 Governance Framework Setup

The Architecture Review Board has six voting members: the Chief Architect (chair, overall architecture integrity and principle enforcement), the VP Engineering for Platform (core services, dispatch, integration platform), the VP Engineering for Payments (payments, reconciliation, financial controls), the Head of Data (data architecture, residency, analytics), the Head of Security & Compliance (security architecture, regulatory and data-protection compliance), and a rotating Regional Expansion Lead — one per active market — representing market-specific regulatory and operational constraints. Quorum is four of six voting members, including the Chief Architect or a delegated deputy. Non-voting standing attendees include the Program Manager, a delivery-team representative for each item under review, and Finance for decisions with material budget impact.

The ARB operates on four cadences: a weekly, 60-minute design review for proposals below the threshold requiring a full architecture contract, with minutes published within 24 hours; a bi-weekly, 90-minute program governance review covering roadmap progress, open architecture risks, and escalations; an ad hoc exception review, convened within 3 business days of a delivery team requesting a deviation from an approved standard or principle; and a quarterly principle review, in which the ARB re-examines the ten principles against how the program has actually unfolded and formally amends or retires a principle if evidence warrants — principles are living governance instruments here, not a document frozen at kickoff.

The escalation path runs in four steps. Most day-to-day architecture questions are resolved between a delivery team and its assigned architecture liaison without a full ARB session. Unresolved or cross-team-impacting questions go to the weekly design review first. A decision escalates to the full ARB (bi-weekly or ad hoc) when it trades off an approved principle, carries budget impact above \$150,000 illustrative, or is contested between two VP-level stakeholders. The Executive Steering Committee is reserved for decisions that change program scope, timeline, or the approved business case — the ARB escalates to this committee, it does not resolve scope disputes on its own authority. An escalation is never silently dropped: every item raised to the ARB receives either a decision, a deferral with a named owner and date, or a formal escalation upward within one review cycle.

The ARB enforces the principles through three mechanisms: a design review gate (no solution building block proceeds to delivery without an ARB sign-off naming which principles it satisfies and which, if any, it deviates from and why), architecture contracts (every delivery team engagement is bound by a contract referencing specific principles and standards, with non-compliance a contract breach subject to the escalation path, not an informal conversation), and an exception register (every approved deviation is logged with its rationale, approver, and an expiry or review date, so temporary exceptions do not silently become permanent architecture). Full detail on how this governance is exercised during delivery — compliance checkpoints, the contracts process, and non-compliance handling — is in Phase G, Section 9.

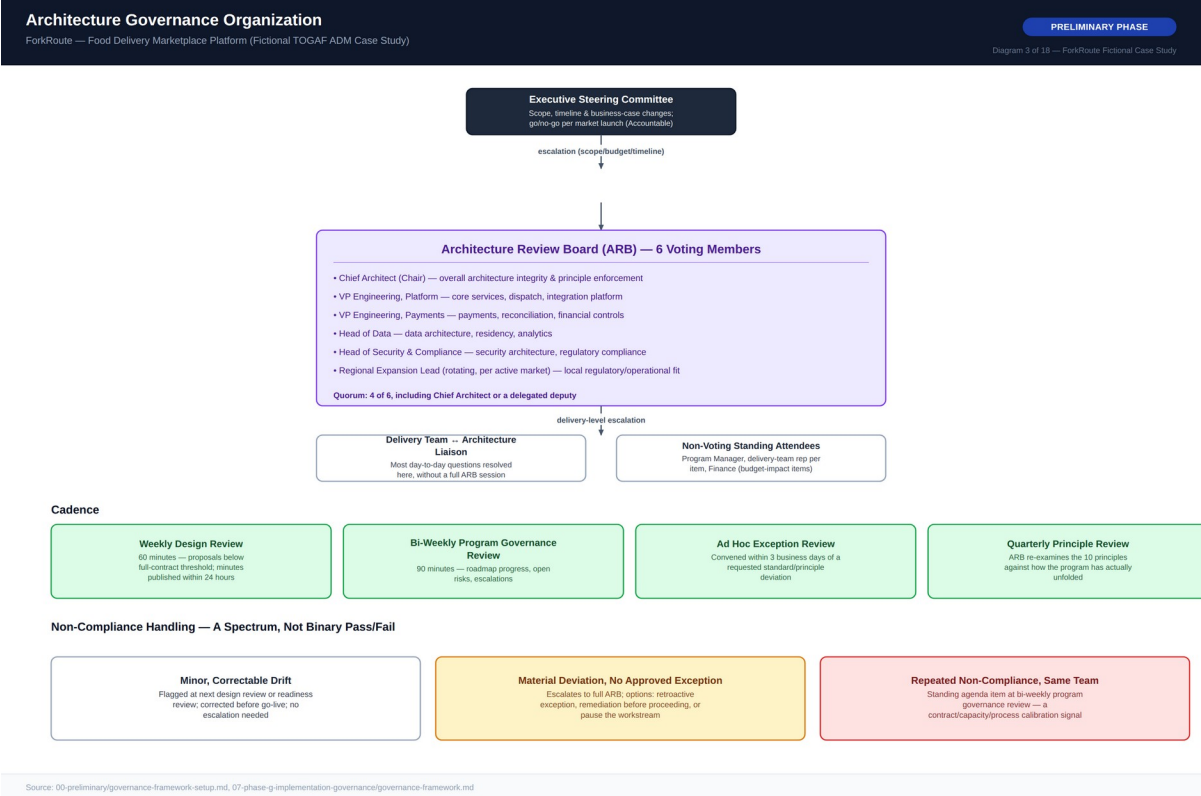


Diagram 3 — Architecture Governance Organization

3. Phase A — Architecture Vision & Scope

3.1 Problem Statement & Target-State Vision

Continuing to expand on the current architecture would mean repeating the current 8–10 week per-market dispatch build four more times, inheriting the integration platform's single-point-of-failure risk at roughly 2.5x the current partner count, and either delaying real-time payout capability in every new market or building it as another one-off. ForkRoute's target architecture instead treats market expansion as a configuration exercise, not an engineering project. A single, config-driven dispatch and allocation engine loads market-specific rule sets at runtime; a new market ships when its configuration bundle is authored and validated, not when new code is deployed. An event-driven payments backbone maintains payment and payout state as an append-only, replayable log, enabling real-time payout and live surge-pricing calculation in every market from day one. A multi-tenant restaurant integration platform isolates each partner's traffic and failure blast radius, so no single partner incident can degrade the platform for others. And data residency is a first-class architectural concern, with jurisdiction-aware storage and processing built in rather than retrofitted market by market.

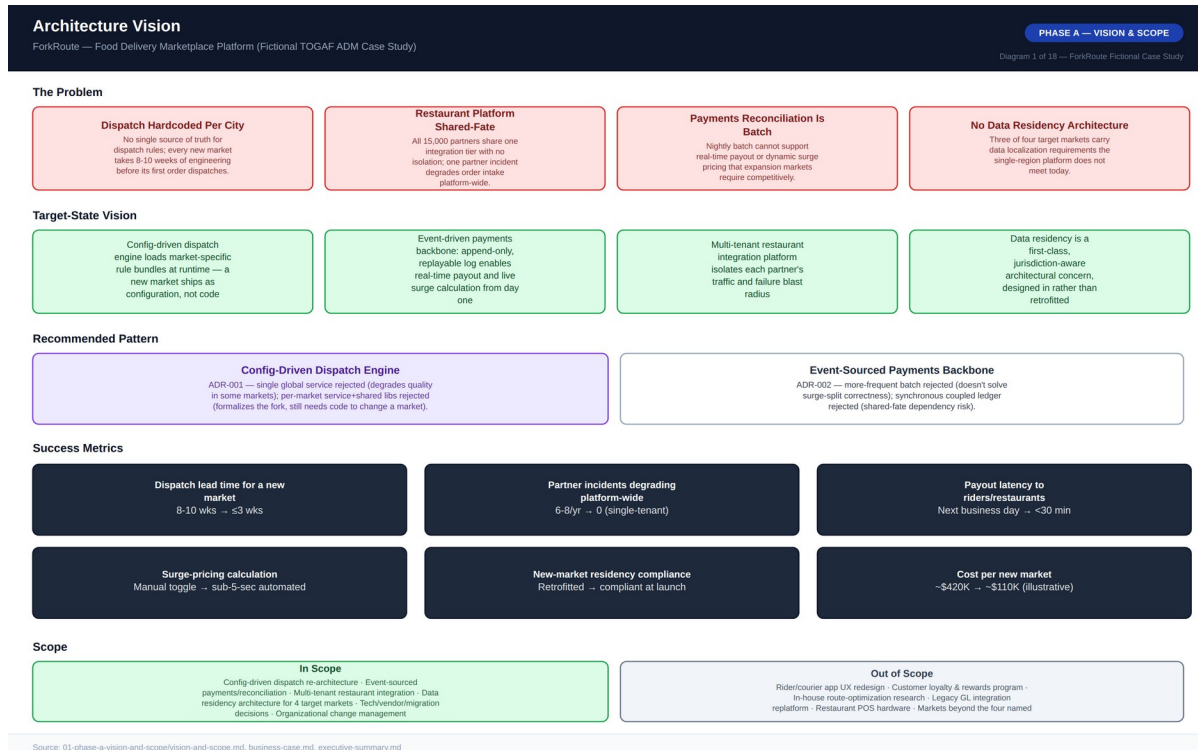


Diagram 1 — Architecture Vision One-Pager

3.2 Scope Boundaries

In scope: re-architecture of the order-dispatch and driver-allocation engine to a config-driven model (Principle 1, Principle 5); re-architecture of payments and reconciliation to an event-sourced model supporting real-time payout and surge pricing (Principle 3); re-architecture of the restaurant partner integration platform for multi-tenant isolation (Principle 2); the data architecture changes required to support jurisdiction-aware residency in the four target expansion markets (Principle 4); the technology,

vendor, and migration decisions required to deliver the above across the home market and the four expansion markets; and organizational change management for the engineering, operations, and partner-facing teams affected by the above.

Explicitly out of scope, each evaluated and excluded to keep the 18-month timeline credible: the rider/courier mobile application UX redesign — under active discussion by the Product organization but not a dependency of the dispatch, payments, or integration re-architecture, and bundling it would extend timeline without changing the underlying architecture risk this program addresses; the customer-facing loyalty and rewards program — a business initiative with its own roadmap that consumes the rebuilt payments platform's APIs but is not designed as part of this engagement; in-house last-mile route-optimization algorithm research — the vendor evaluation in Phase E treats route optimization as a build-vs-buy decision explicitly to avoid the program absorbing a multi-year algorithms research effort; full replatforming of the home-market payments core's legacy general ledger integration — the GL integration is wrapped, not replaced, in this program (see ADR-002), with a full GL replacement left as a separate, larger finance-systems initiative outside this program's charter and budget; and restaurant partner point-of-sale (POS) hardware — ForkRoute integrates with partners' existing POS systems via API and does not manufacture, resell, or architect POS hardware. New markets beyond the four named in the expansion mandate are also out of scope: the architecture is designed to generalize to further markets, but this engagement's scope, budget, and roadmap are bounded to the four markets named in the expansion mandate — a fifth market is a new business case, not an extension of this one.

3.3 Success Metrics

Metric	Baseline (As-Is)	Target (To-Be)
Engineering lead time to launch a new market's dispatch rules	8–10 weeks	≤ 3 weeks
Restaurant partner incidents causing platform-wide degradation	6–8 per year (illustrative)	0 tolerated platform-wide; contained to single-tenant blast radius
Payout latency to riders/restaurants	Next business day (batch)	Under 30 minutes in supported markets
Time to calculate and apply a surge-pricing adjustment	Not supported dynamically (manual city-level toggle)	Sub-5-second automated recalculation
New-market data residency compliance at launch	Retrofitted post-launch historically	Compliant at launch, by design
Cost per new market to reach dispatch-ready state	~\$420K illustrative	~\$110K illustrative

3.4 Stakeholder Map

Thirteen stakeholder groups were assessed for their concerns and RACI accountability across the program's major decision categories, condensed to eleven for the power/interest quadrant placement in Diagram 2 (Delivery Teams and External Vendors are combined into a single quadrant entry there; the full thirteen-group breakdown, including the four Regional Expansion Leads as a distinct entry, governs the RACI matrix itself). The Chief Architect is Accountable for architecture principles and for the dispatch, payments, and restaurant-integration target architectures, with the relevant VP Engineering Responsible in each case and

delivery teams Responsible for execution. The Head of Data is Responsible for the technical implementation of data residency per market, with Regional Expansion Leads Responsible for confirming local regulatory interpretation and the Chief Architect retaining overall Accountable status to resolve any conflict between the two.

Two RACI assignments were explicitly debated at program kickoff and are recorded for transparency rather than smoothed over. Data residency approach per market was initially proposed as Chief-Architect-accountable throughout, but Regional Expansion Leads argued they carry the direct regulatory relationship in-market and needed shared Responsible status — the ARB agreed to the split described above. Go/no-go per market launch was contested between the CEO, who wanted sole authority, and the Chief Architect, who argued technical readiness needed a named Responsible owner distinct from the business go/no-go decision; the resolution is that the CEO/Executive Steering Committee is Accountable for the business decision to launch, while the Chief Architect is Responsible for certifying technical readiness as an input to that decision, not a veto over it.

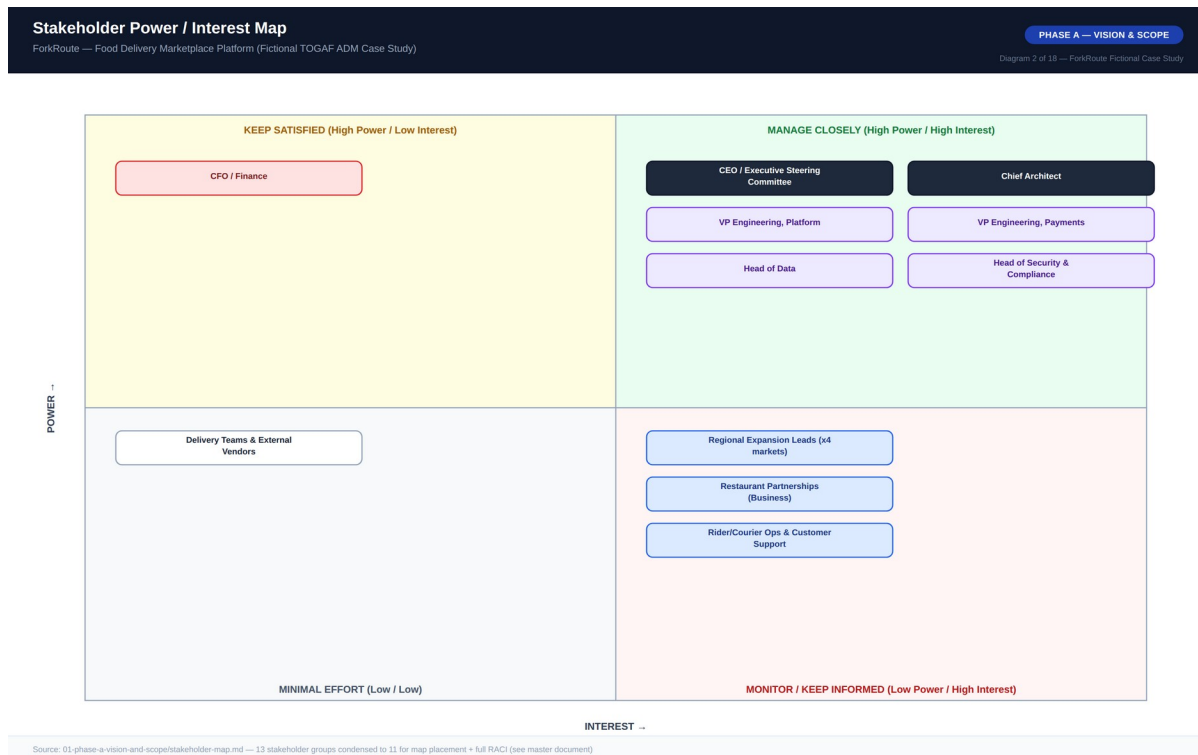


Diagram 2 — Stakeholder Power / Interest Map

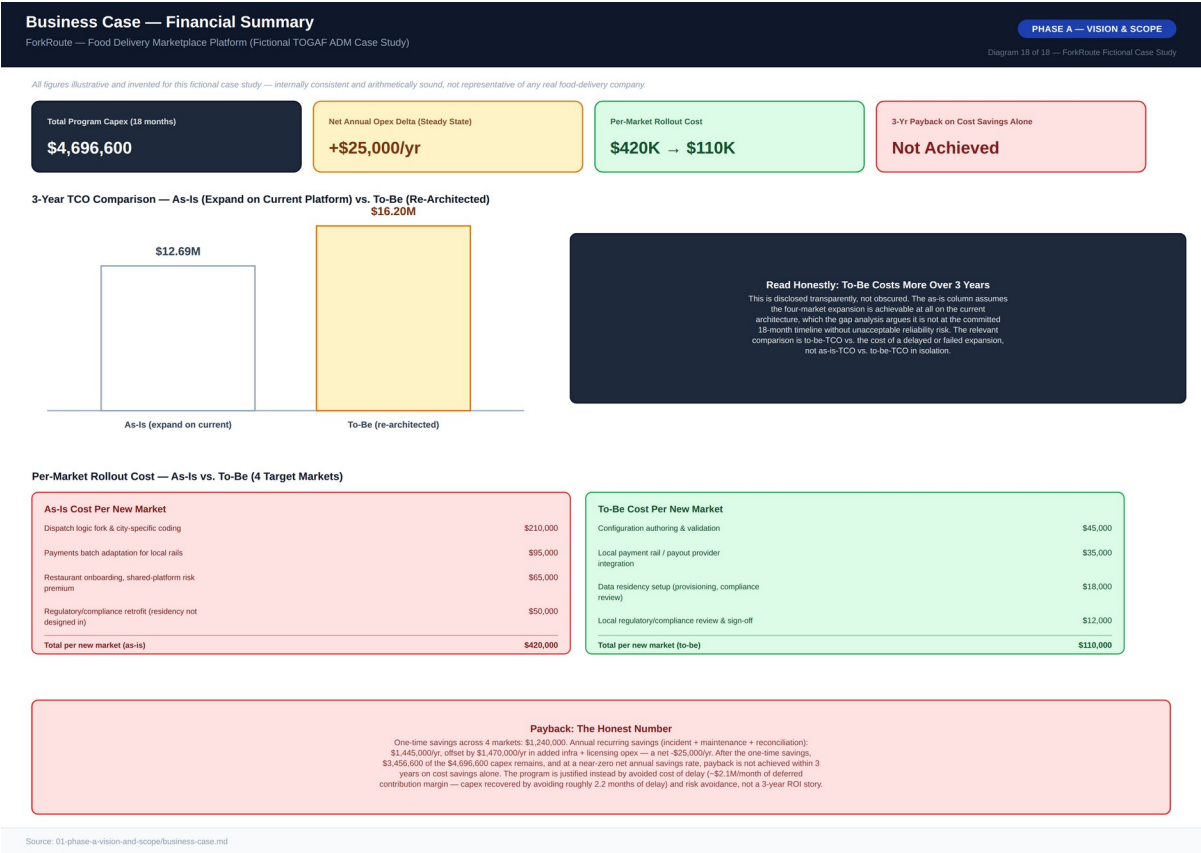
3.5 Business Case Summary

All figures in the business case are illustrative, invented for this fictional case study, and constructed to be internally consistent and arithmetically sound for a marketplace of ForkRoute's stated size — not representative of any real company's financials. Capex totals \$4,696,600 across eight line items (the config-driven dispatch engine at \$756,000/42 person-months; the event-driven payments backbone at \$990,000/55 person-months; the multi-tenant restaurant integration platform at \$684,000/38 person-months; the data residency architecture at \$432,000/24 person-months; vendor evaluation, procurement, and licensing at \$430,000; migration tooling and dual-run infrastructure at \$288,000/16 person-months; change management

and training at \$144,000/8 person-months; and program management and governance overhead at \$360,000/20 person-months), summing to a \$4,084,000 subtotal plus a 15% contingency of \$612,600.

Per-market rollout cost drops from an as-is \$420,000 (dispatch logic fork and city-specific coding \$210,000, payments batch adaptation for local rails \$95,000, restaurant onboarding at a shared-platform risk premium \$65,000, and regulatory/compliance retrofit \$50,000) to a to-be \$110,000 (configuration authoring and validation \$45,000, local payment rail/payout provider integration \$35,000, data residency setup \$18,000, and local regulatory/compliance review \$12,000) — a savings of \$1,240,000 across the four target markets. Steady-state annual opex delta is a near-neutral net +\$25,000/year: +\$850,000 in cloud infrastructure and +\$620,000 in vendor licensing, offset by -\$420,000 in incident-response cost (from 6–8 platform-wide incidents/year at ~\$65K average cost down to isolated, contained incidents), -\$780,000 in dispatch-maintenance labor, and -\$245,000 in reconciliation labor.

Read at face value, the 3-year TCO is higher for the to-be architecture — \$16,196,600 versus \$12,690,000 as-is — and this is disclosed transparently rather than obscured, because the as-is column assumes the four-market expansion is achievable at all on the current architecture, which the gap analysis in Phase E argues it is not at the committed 18-month timeline without unacceptable reliability risk. Using only directly quantifiable savings streams, the one-time rollout savings of \$1,240,000 plus \$1,445,000/year in recurring incident, maintenance, and reconciliation savings are offset by \$1,470,000/year in added infrastructure and licensing opex — a net -\$25,000/year — leaving \$3,456,600 of the \$4,696,600 capex unrecovered by cost savings within the 3-year window. Payback on cost savings alone is therefore not achieved within 3 years, and this is the number presented to the Executive Steering Committee rather than an inflated ROI figure. The investment is justified instead by avoided cost of delay (an illustrative \$2.1M per month of deferred contribution margin per month of expansion delay, meaning program capex is recovered by avoiding roughly 2.2 months of delay) and by risk avoidance — a platform-wide outage risk in new markets where ForkRoute has no brand-trust cushion, which the Executive Steering Committee judged unacceptable to carry forward unchanged.



4. Phase B — Business Architecture

4.1 As-Is Business Architecture

When a customer places an order, dispatch determines which available rider is offered the delivery using a mix of proximity, rider rating, and current load. This logic exists as city-specific code paths within the dispatch service: the home market's platform grew city by city, and each city's operations team, wanting control over local behavior (delivery radius, batching of multiple orders per rider, surge activation), got that control by having engineering hand-code it. There is no single authoritative rule set — there are, in practice, as many rule variants as there are cities with meaningfully different logic, and no registry of which cities run which variant. Surge pricing, where it exists at all, is a manual operational toggle a city operations manager flips during a demand spike, applying a flat citywide multiplier rather than a calculated, granular adjustment, and it is not tied to any live view of driver supply.

All 15,000 restaurant partners integrate through a single shared integration tier — one set of services handling menu sync, order injection, and status updates for every partner regardless of size or traffic pattern. There is no per-partner resource isolation: a large chain's menu-sync job or a traffic spike from one partner's promotional campaign runs in the same shared infrastructure as a single independent restaurant's order flow, with no circuit breaker or throttle preventing one partner's load from degrading another's experience or the platform's overall order intake. Onboarding a new partner is semi-manual, with a partnerships team member configuring the partner in an admin console and a solutions engineer typically assisting with menu-format mapping for anything outside the standard template — adequate at current volume, but not something that scales cleanly to roughly 2.5x the partner count four new markets imply.

Money movement — customer charge, restaurant payout, rider payout, platform commission — is tracked across transactional tables that are the operational system of record during the day, but authoritative reconciliation happens as a nightly batch job that aggregates the day's transactions, applies commission and fee logic, and produces payout instructions submitted to banking partners the following business day. A restaurant or rider who completed work today does not see funds move until at least the next business day, discrepancies surface only once daily rather than as they occur, and there is no mechanism to calculate or apply a dynamic surge adjustment that flows correctly through to payout splits in real time — the batch model assumes each day's pricing was static and known in advance.

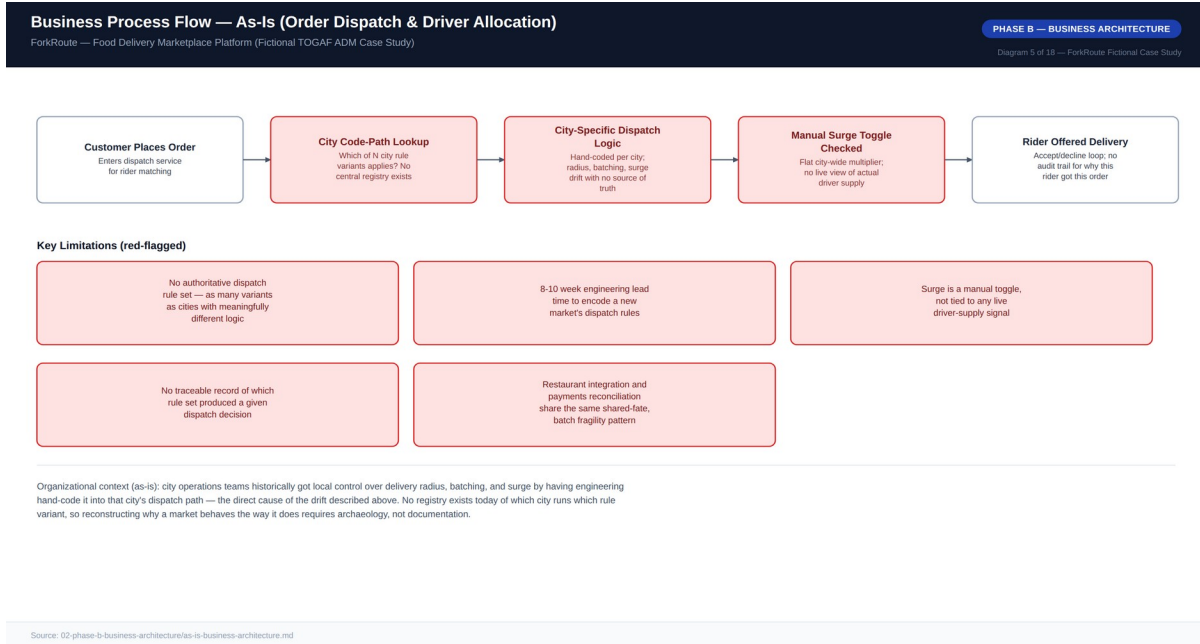


Diagram 5 — Business Process Flow, As-Is (Order Dispatch & Driver Allocation)

4.2 To-Be Business Architecture

A single dispatch engine evaluates every order against a market-specific configuration bundle — dispatch radius, batching rules, rider-matching weights, and surge parameters — loaded at runtime rather than compiled into code (ADR-001). A new market or a rule change is authored, validated against a schema and a simulation harness, and published; no service redeployment is required. Surge pricing becomes a calculated function of live driver-supply and demand signals within each market's configured bounds, not a manual citywide toggle, and it feeds directly into the event-driven payments backbone so a rider's or restaurant's payout for a surge order reflects the correct split in real time (ADR-004).

The integration platform becomes explicitly multi-tenant: each restaurant partner, or partner group for chains, is provisioned with isolated rate limits, dedicated processing lanes, and independent circuit breakers, so one partner's traffic anomaly or integration defect degrades only that partner's flow (ADR-003). Onboarding is streamlined through self-service menu-format templates for the majority case, with solutions-engineer assistance reserved for genuinely nonstandard integrations — a deliberate reallocation of scarce solutions-engineering time toward the partners who actually need it. Every financial event — charge authorization, capture, commission calculation, payout instruction, dispute — is written to an append-only, replayable event log that is the authoritative system of record (ADR-002). Reconciliation becomes continuous rather than nightly batch: discrepancies are detected within minutes, not up to 24 hours later, and payout instructions can be generated and submitted in near-real time, supporting the sub-30-minute payout target.

Two operational changes ride alongside the technical re-architecture. City operations managers lose direct manual control over surge pricing in favor of the automated engine — a genuine authority change for that role, not just a tooling change, deliberately called out as an adoption risk rather than glossed over (see Section 10). And the partnerships and solutions-engineering teams shift from uniform hands-on onboarding to a triage model, spending relatively more time on complex partners and relatively less on the standard case, which changes how their workload and staffing model should be planned. What does not change: the customer-facing ordering flow, the restaurant-facing order-acceptance flow, and the rider app's delivery-

acceptance flow are not being redesigned in this program — the to-be architecture changes what happens behind those interfaces, not the interfaces customers, restaurants, and riders directly interact with.



Diagram 6 — Business Process Flow, To-Be (Order Dispatch & Driver Allocation)

4.3 Business Capability Map

Twelve business capabilities were rated on a 1–5 maturity scale (1 = ad hoc/manual, 5 = optimized/self-improving), as-is versus target. The three largest gaps — Dynamic Pricing/Surge Management, Restaurant Integration Resilience, and Payments Reconciliation, all rated 1→4 — map directly to the three structural problems named in the vision and scope. This is intentional: the capability map is used as a diagnostic confirming the program's technical scope targets the capabilities with the largest maturity gaps, rather than the capabilities that happen to be easiest to improve. Data Residency & Jurisdiction Compliance is a fourth major gap (also 1→4) and is the one most directly tied to expansion-market legal requirements rather than home-market operational pain, which is why it did not surface as an urgent problem until the expansion mandate made it one. Architecture Governance and Change Management & Adoption are deliberately rated to lower target maturities (4 and 3 respectively for governance, 3 for change management) — the program's primary mandate is the technical platform, and over-investing in process maturity beyond what the program itself needs would divert budget from the capabilities the business case is funded to fix.

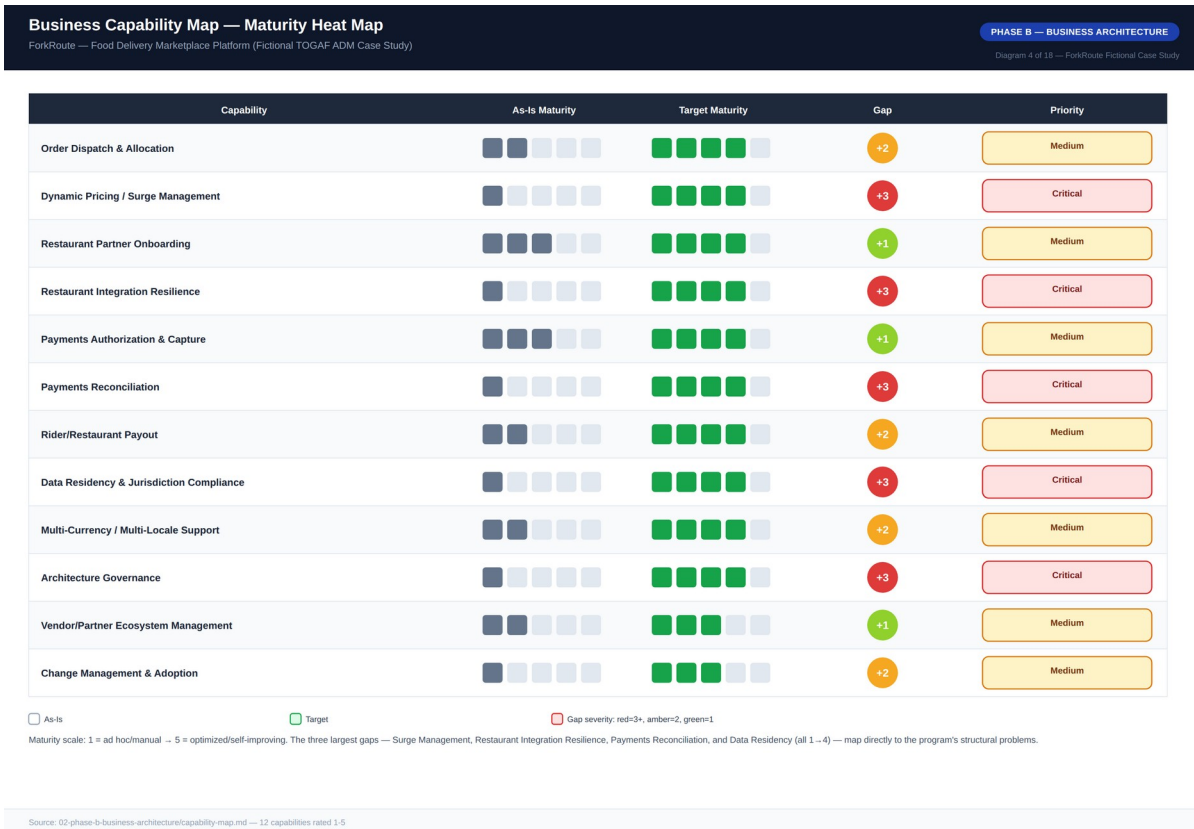


Diagram 4 — Business Capability Map, Maturity Heat Map

4.4 Business Footprint

Six organizational units carry the program's business footprint. Platform Engineering owns dispatch and order orchestration — config-driven dispatch, market bundle authoring and validation, and dry-run simulation — supported by the Config-Driven Dispatch Engine and Market Config Store on managed container orchestration with a schema-validated config store. Payments Engineering owns payments and reconciliation — event-sourced payment state, continuous reconciliation, and payout orchestration — on a durable event log plus the LedgerBridge bought platform. Data & Residency owns jurisdiction-aware data architecture — zone assignment, infrastructure-level enforcement, and de-identified cross-zone export — on cloud-provider regional residency features plus network/IAM boundary enforcement. Security & Compliance owns regulatory and audit compliance — config bundle audit trail, PII/jurisdiction tagging, dispute evidence — via structured, correlation-ID-propagated logging and tracing. Restaurant Partnerships owns partner onboarding and experience — self-service onboarding for standard cases, triage for nonstandard integrations — on a tenant-scoped API gateway with per-tenant quotas. Architecture & Governance owns program governance — ARB review, architecture contracts, exception register, principle enforcement — on off-the-shelf documentation and workflow tooling, consistent with Principle 7's rejection of bespoke governance tooling as a differentiation investment.

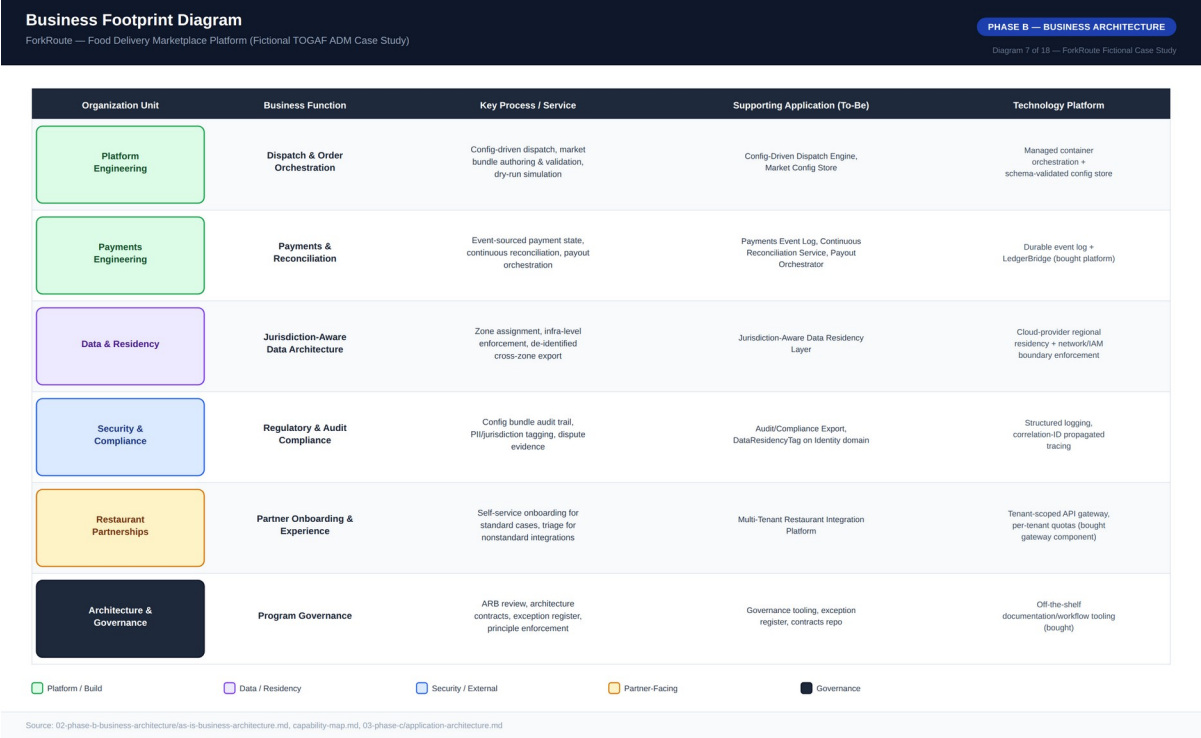


Diagram 7 — Business Footprint Diagram

5. Phase C — Information Systems Architecture

5.1 Application Architecture

The current platform is best described as a modular monolith with city-coded branches: a single deployable dispatch service contains conditional logic keyed on city ID, a single restaurant-integration service handles all partner traffic without tenant separation, and a payments service writes to transactional tables that a separate nightly batch job reads to produce reconciliation output. These are three separately deployed services, each internally monolithic with respect to the dimension that now needs to vary — city, tenant, and settlement timing respectively.

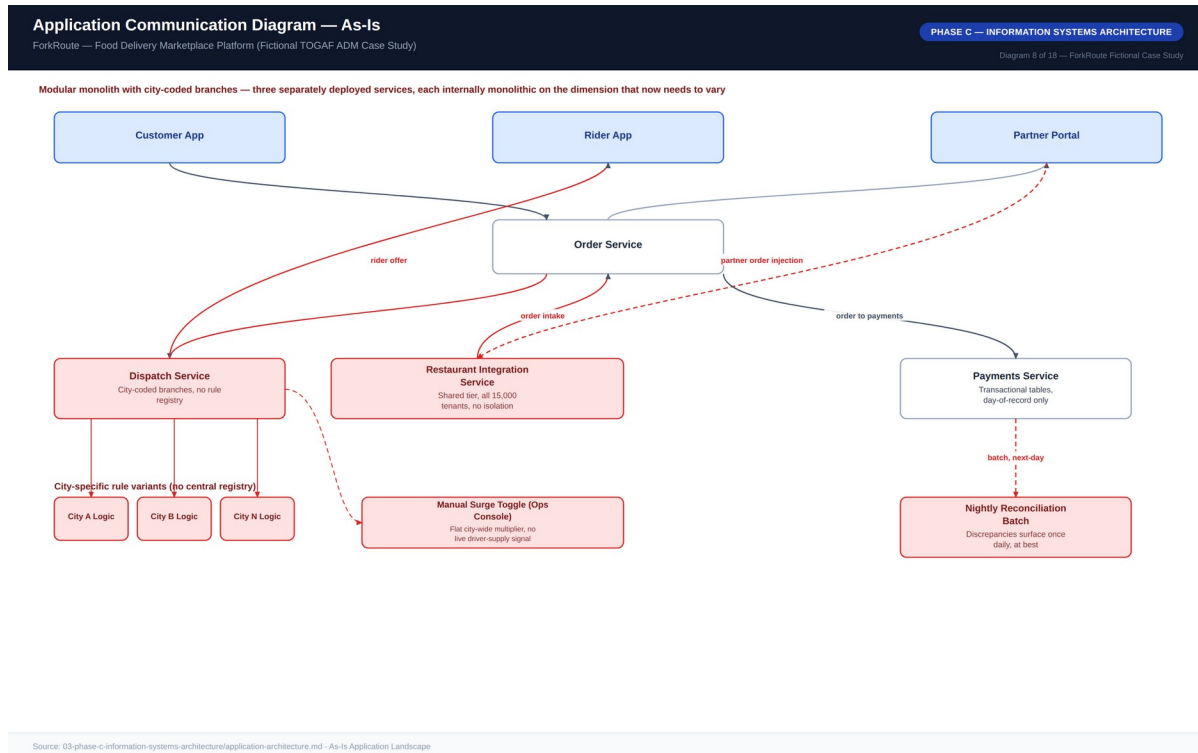


Diagram 8 — Application Communication Diagram, As-Is

The target architecture decomposes the three problem areas into services organized around the axis that must vary independently — market configuration for dispatch, tenant for integration, and event stream for payments — while deliberately not decomposing further than that. The program is not pursuing a maximalist microservices decomposition: Order Service and Rider Service remain unchanged, evaluated against the same "does this need to vary by market or tenant independently of the rest of the platform" test applied to dispatch, integration, and payments, and both failed that test. Decomposing them anyway, on the theory that more services is more architecturally mature, was explicitly rejected by the ARB as scope creep that would consume engineering capacity without addressing a gap identified in the capability map. Key decisions reflected in the to-be topology: the Dispatch Engine is a single deployable across all markets, with per-market behavior living entirely in the Market Config Store (Principle 1, ADR-001); the Restaurant Integration Platform is explicitly multi-tenant, with tenant isolation an architectural property of the service itself rather than bolted-on rate limits (Principle 2, ADR-003); and the Payments Event Log is the system of record, with

Continuous Reconciliation and the Payout Orchestrator as downstream consumers — the most significant topology change from as-is, replacing "batch job reads transactional tables" with "services subscribe to an event stream" (Principle 3, ADR-002).

Four integration patterns govern the to-be topology, each with a rejected alternative on record. Event-carried state transfer (async pub/sub) is used for payments-state propagation to reconciliation, payout, and analytics; synchronous request/response from each consumer to the payments service was rejected because it would create tight coupling and a shared-fate dependency the moment any one consumer is slow — the opposite of the isolation the program is trying to achieve. Config-as-data with runtime evaluation is used for dispatch rule variation per market; feature-flag-per-city inside a single code path was rejected because flags don't scale past a handful of binary toggles, and the dispatch rule set has too many interacting parameters to express safely as flags. A tenant-scoped API gateway with per-tenant quotas is used for restaurant partner traffic isolation; a single gateway with global rate limiting only was rejected because it protects the platform in aggregate but does nothing to stop one noisy tenant from starving another's quota within the global ceiling. And synchronous request/response is retained deliberately for order placement and the dispatch offer to the rider app, since both have a hard user-facing latency expectation — a case where the program deliberately did not apply Principle 3's event-sourcing bias outside the payments domain it was written for.

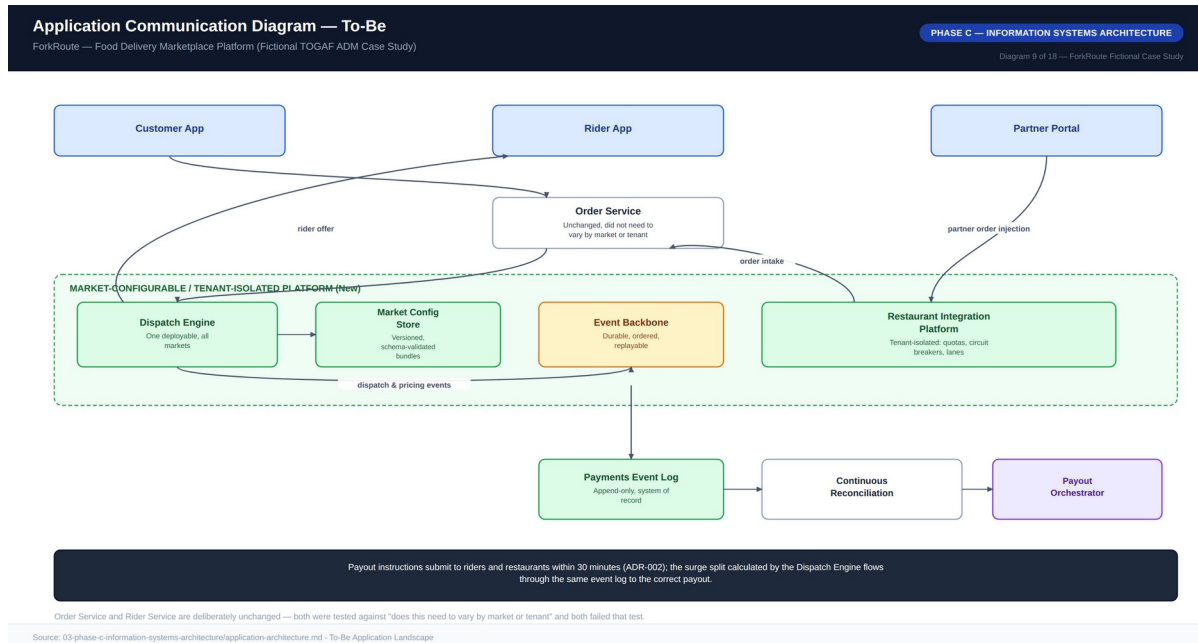


Diagram 9 — Application Communication Diagram, To-Be

5.2 Data Architecture

Seven core data domains span the platform: Order (Order, OrderLine, DeliveryAddress — unchanged, transactional); Dispatch (DispatchOffer, RiderCandidate, MarketConfigBundle — as-is embedded in city-coded logic with no distinct entity, to-be owned by the Dispatch Engine and a versioned config store); Rider (Rider, RiderShift, RiderLocation — unchanged system of record, with location additionally published as events); Restaurant Partner (Restaurant, MenuItem, PartnerContract — shared DB as-is, tenant-partitioned to-be); Payment (PaymentAuthorization, Capture, Commission, PayoutInstruction — transactional DB plus nightly batch output as-is, an append-only event log plus materialized views to-be); Pricing (BaseFare, SurgeFactor, PriceQuote — not modeled as a distinct entity as-is, a first-class computed and logged

SurgeFactor entity to-be); and Identity/Compliance (CustomerProfile, KYCRecord, DataResidencyTag — a single global customer table as-is, partitioned and tagged by jurisdiction to-be).

Today, data lives in market-agnostic, mostly single-region transactional databases, with payments reconciliation adding a batch-derived reporting layer on top — there is no concept of a jurisdiction tag on customer or transaction records, because until now a single legal jurisdiction has always applied. The target topology introduces jurisdiction as a first-class partitioning dimension: each of the four expansion markets is assigned to a residency zone (Home Market, Market A, and Markets B/C/D grouped, per the diagrams in this document), and customer, rider, and restaurant PII along with transactional payment data is stored and processed within that zone, enforced at the infrastructure level — network and IAM boundaries — not application-level filtering alone (ADR-005). Cross-market aggregate reporting runs from de-identified or aggregated daily exports pushed to a central analytics zone, not live cross-region queries; a live federated cross-region query layer was rejected primarily on residency-compliance grounds (it makes it easier, not harder, to accidentally construct a query that crosses a jurisdiction boundary) and secondarily on latency and operational-complexity grounds.

The MarketConfigBundle entity is itself treated as governed master data: every version is immutable once published, changes go through the same schema validation and ARB-adjacent review as any other production change, and the dispatch engine always logs which config version handled a given order — giving full auditability over which rule set produced which dispatch decision. This traceability is a direct requirement from the Head of Security & Compliance stakeholder group, who needed a defensible answer to "why did this rider get this order" for dispute handling. Because config is published, not mutated in place, there is no need for change-data-capture to detect drift — the version itself is the audit record.

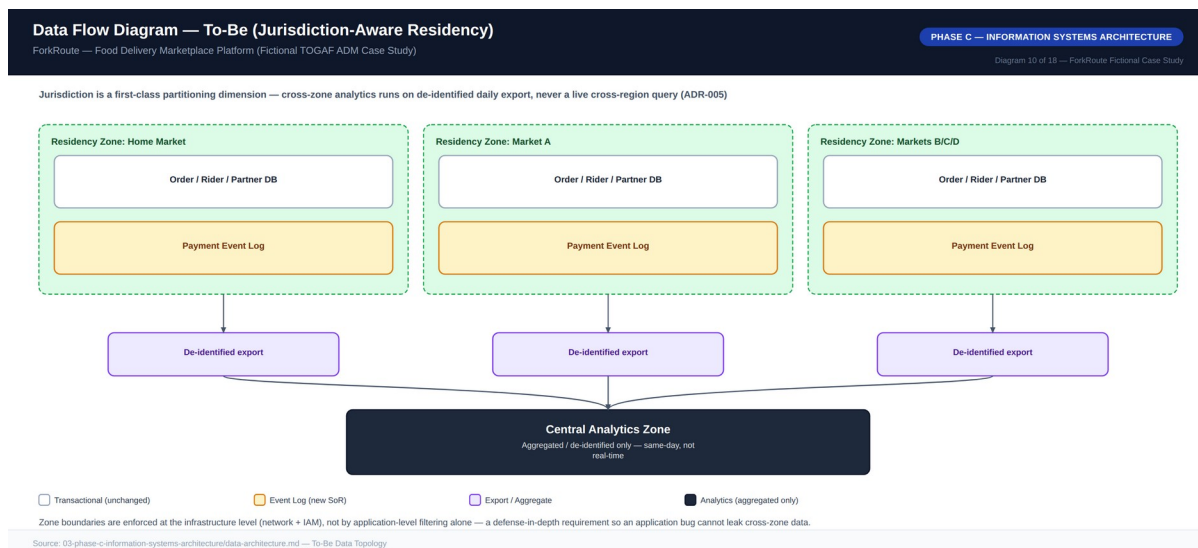


Diagram 10 — Data Flow Diagram, To-Be (Jurisdiction-Aware Residency)

5.3 Data Ownership / Domains

Order and Rider remain owned by their unchanged transactional databases, since order and rider management do not need to vary by market or tenant the way dispatch, integration, and payments do. Dispatch is newly owned by the Dispatch Engine plus a versioned config store, with the MarketConfigBundle treated as master data. Restaurant Partner ownership moves from a shared database to a tenant-partitioned one under the multi-tenant integration platform. Payment ownership moves from a transactional database



6. Phase D — Technology Architecture

6.1 Reference Architecture Pattern

The core technology pattern underpinning both the dispatch engine and the payments/reconciliation model is an event backbone — a durable, ordered, replayable log — sitting between order/dispatch/pricing producers and the consumers that need to react to state changes: Continuous Reconciliation, the Payout Orchestrator, Analytics/Reporting, Support Tooling, and Audit/Compliance Export. Producers append events; they do not call consumers directly, and consumers subscribe to the streams relevant to them and maintain their own materialized view of state, rebuildable by replaying the log. The pattern is appropriate when all of four conditions hold: multiple independent consumers need the same underlying state changes; an audit trail of what happened, in what order, has intrinsic value; consumers can tolerate eventual consistency on the order of low seconds; and the team owns the operational maturity to run a durable log — schema governance, consumer-lag monitoring, replay tooling, and dead-letter handling being real, ongoing operational overhead, not a one-time setup cost.

This is stated as a reusable reference pattern with explicit anti-cases, because event-driven architecture is frequently over-applied past the point where it earns its complexity. The ARB rejected using this pattern in three places during the program's design. Customer-facing order confirmation needs a synchronous, sub-2-second response with exactly one consumer (the customer app) that needs the answer now — routing it through an async backbone would add latency and failure modes with no corresponding benefit, so synchronous request/response is used instead. The rider app's "is this delivery still assigned to me" check during an active delivery is a strongly-consistent read against current state, not a candidate for eventual-consistency-tolerant event replay — a direct query against the owning service is used instead. And the restaurant partner onboarding workflow has one producer and effectively one consumer, happening tens of times per day rather than thousands of times per second — standing up event infrastructure here would be solving a scaling problem that does not exist, so a straightforward transactional write with an application-level notification is used instead. Where strict, immediate cross-entity consistency is a hard financial requirement the event backbone cannot itself satisfy — the guarantee that a payout can never be instructed against a payment that has not yet cleared authorization — that invariant is enforced synchronously inside the Payments Service before the corresponding event is even emitted; the backbone is downstream of the invariant, never its sole enforcement mechanism.

Resilience is differentiated by consumer, per Principle 10: the Payout Orchestrator carries a materially tighter alerting threshold (under 2 minutes of consumer lag) than Analytics (up to 4 hours tolerable). Replay is a first-class operational capability, exercised routinely — at minimum quarterly — rather than assumed to work only in an emergency, and schema evolution follows Principle 6: a breaking event-schema change requires a new event-type version published alongside the old for a defined deprecation window, never an in-place field-meaning change.

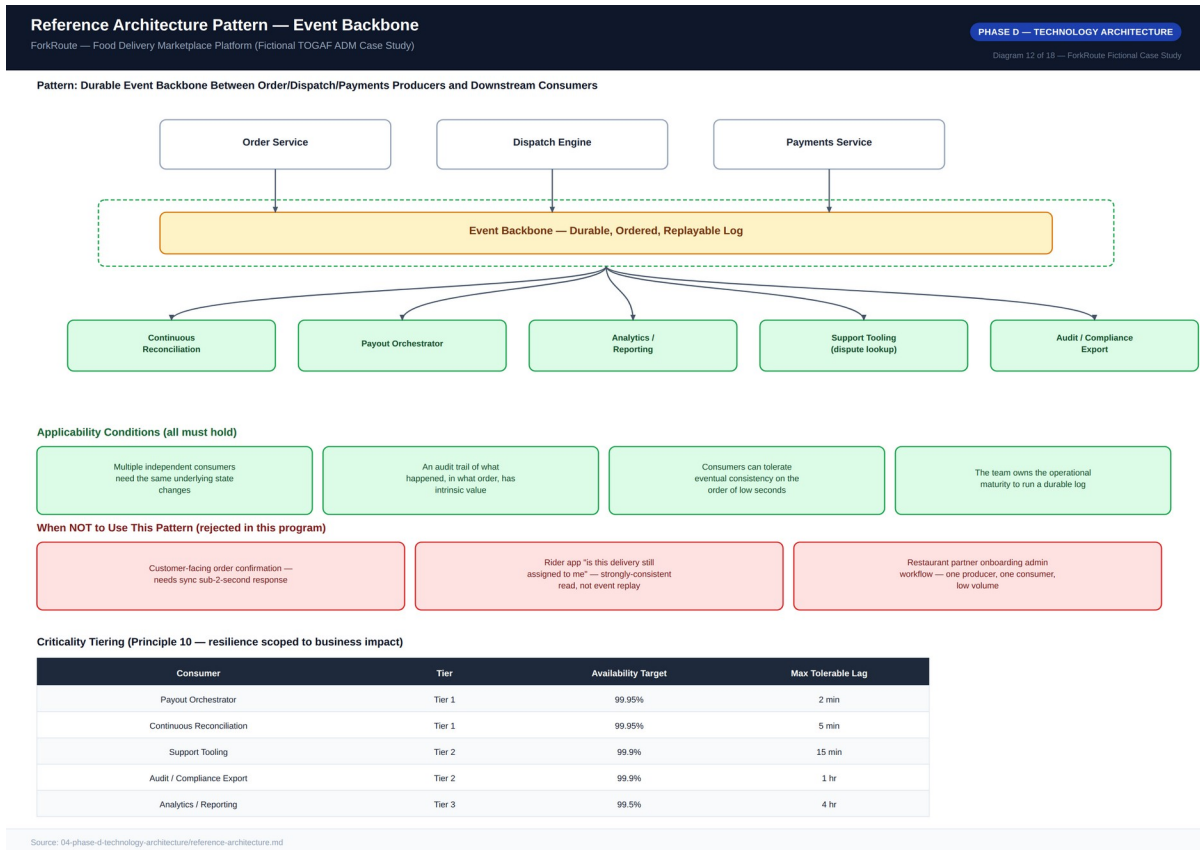


Diagram 12 — Reference Architecture Pattern

6.2 Technology Standards

The approved stack spans platform and infrastructure standards (a durable, partitioned, replayable log with at-least-once delivery and consumer-group semantics for the event backbone, chosen as a category standard rather than a specific vendor product since this is a build-on-open-standard layer; a versioned, schema-validated document store with strong read consistency for the market config bundles, since a rider must never be dispatched against a stale rule set; ACID-compliant relational stores, one logical instance per residency zone, for entities not event-sourced; an append-only, immutable, cryptographically checksummed payments event log; and centralized secrets management, since config bundles are treated as lower-trust, frequently-changing artifacts that must never become a path to credential exposure), identity and compliance standards (infrastructure-level network and IAM boundary enforcement per residency zone rather than application-level filtering alone, a defense-in-depth requirement so an application bug cannot leak cross-zone data even if it tries; fixed-point decimal representation with an explicit ISO currency code on every monetary field and no floating-point monetary arithmetic anywhere in the payments domain; and structured, correlation-ID-propagated logging with distributed tracing across the dispatch-to-payout path, needed to debug a single order's journey across the decomposed to-be services), and integration standards (schema-first, versioned, contract-tested-in-CI service-to-service APIs). City- or market-keyed conditional branches for any behavior the Market Config Store can express, shared unbounded rate limits across restaurant partner tenants, and floating-point types for monetary fields are all explicitly deprecated and disallowed.

Deviating from a standard is a real decision that must be argued, not a form to route around governance. A delivery team submits an exception request naming the specific standard, the technical reason, the alternative proposed, and — critically — an expiry or review date; a request with no review date is returned unprocessed. Low-impact, single-team-scoped exceptions may be approved at the weekly design review by the Chief Architect or a delegated liaison; exceptions with cross-team impact, budget impact above the \$150,000 threshold, or that touch a Tier 1 capability escalate to the full ARB. Every approved exception is logged in the exception register with its approver, rationale, and review date; at that date it is retired, renewed with updated justification, or — if it has proven durable and broadly applicable — proposed as an amendment to the standard itself. One approved exception is currently on record: during the payments backbone build, the Payments engineering team requested a scoped exception to run a secondary, synchronous fallback path for payout instruction submission to one specific expansion market's banking partner, whose API does not support the asynchronous webhook confirmation pattern the standard event-driven payout flow assumes. The ARB approved this as a market-specific exception, not a change to the standard, with a 12-month review date, on the basis that the banking partner's roadmap indicated async support was planned and re-architecting the payout orchestrator to accommodate one non-conforming partner permanently was not justified against a temporary integration gap.

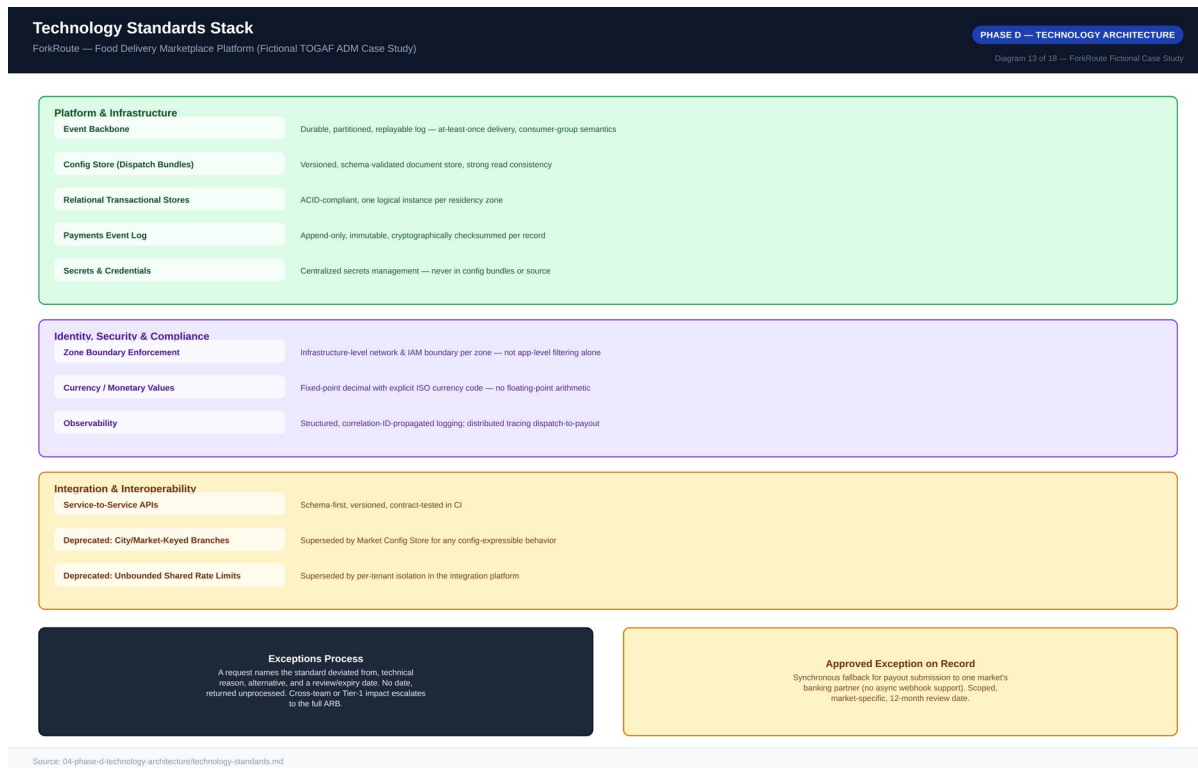


Diagram 13 — Technology Standards Stack

7. Phase E — Opportunities & Solutions

7.1 Gap Analysis

Twelve gaps were registered, each scored on Business Impact (1–5, how directly it blocks the expansion mandate or creates ongoing operational risk) and Closure Complexity (1–5, engineering and organizational effort to close), and prioritized using $\text{Impact} \div \text{Complexity}$ as a rough sequencing signal — high-impact, lower-complexity gaps are closed first where dependencies allow, but a hard dependency overrides the raw score when the two conflict. Three gaps are foundational and high-impact: dispatch logic hardcoded per city with no config layer (Impact 5, Complexity 4); payments reconciliation as nightly batch rather than event-driven (Impact 5, Complexity 4); and no jurisdiction-aware data residency architecture (Impact 5, Complexity 3). Critically, Gap 5 (data residency) carries a hard dependency relationship with Gap 3 (payments) and with expansion-market work generally: building the payments event log before the residency zone architecture exists would mean re-partitioning a live financial data store later, a materially more expensive and riskier operation than designing the partition in from the start. This is why the migration roadmap sequences the data residency layer ahead of, not alongside, the payments backbone build, even though the two gaps' raw priority-signal scores alone would suggest they could run in parallel.

Restaurant integration platform isolation (Gap 4, Impact 4, Complexity 3) is deliberately sequenced as a parallel workstream rather than serialized after Gaps 1, 3, and 5, because the ARB determined it has no meaningful technical dependency on the dispatch or payments work — the delivery teams and codebases involved are largely disjoint, and serializing it would extend the timeline without a corresponding risk-reduction benefit. The lowest-priority gap on the list, rider/courier mobile app payout-status visibility (Impact 2, Complexity 2), is explicitly excluded from this program's roadmap, consistent with the UX-redesign exclusion in scope: the underlying payout data will exist and be queryable once the Payments Event Log is built, making it a comparatively cheap follow-on project for the Product organization once this program's platform work is complete, not something that needs to be pulled into this program's scope to be technically feasible later.

7.2 Solution Building Blocks

Ten solution building blocks decompose the target capabilities identified in the capability map, each applying Principle 7 (build only what differentiates ForkRoute). Building blocks were identified by taking each capability with a maturity gap of two or more points and asking what the smallest set of independently deployable, independently ownable components is that closes it. Two decomposition alternatives were considered and rejected: a single "Marketplace Platform" mega-block covering dispatch, pricing, and payments together, rejected because it would recreate exactly the kind of tightly coupled, hard-to-reason-about system the program is trying to move away from, and because the three domains have different criticality tiers arguing for independent operational lifecycles; and decomposing down to the individual microservice level (separate SBBs for rider ranking, radius calculation, batching logic within dispatch), rejected as too granular for architecture-level tracking — that level of decomposition is an implementation decision for the delivery team, not something the ARB needs to govern block-by-block.

The Config-Driven Dispatch Engine, Market Config Store & Validation Harness, Dynamic Surge Pricing Calculator, and Multi-Currency Financial Primitives Library are built in-house — each a core differentiator with no vendor product encoding ForkRoute-specific marketplace dynamics, pricing logic, or financial discipline. The Route Optimization/ETA Engine and Architecture Governance Tooling are bought —

commodity capability where building in-house would consume scarce engineering capacity better spent on differentiating work. The Multi-Tenant Restaurant Integration Platform, the Payments Event Log & Continuous Reconciliation Service, the Payout Orchestrator, and the Jurisdiction-Aware Data Residency Layer are hybrid: tenant-isolation logic, reconciliation business logic, orchestration logic, and zone architecture respectively are ForkRoute-specific and built in-house, while the underlying gateway infrastructure, event-streaming infrastructure, funds-movement platform, and regional cloud-provider residency features are bought. The Jurisdiction-Aware Data Residency Layer and the Multi-Currency Financial Primitives Library are foundational dependencies for nearly every other block, which is why both are sequenced early in the migration roadmap rather than built in parallel with the blocks that depend on them.

Solution Building Blocks — Buy / Build / Hybrid		
ForkRoute — Food Delivery Marketplace Platform (Fictional TOGAF ADM Case Study)		
PHASE E — OPPORTUNITIES & SOLUTIONS		
Diagram 14 of 18 — ForkRoute Fictional Case Study		
Solution Building Block	Buy / Build / Hybrid	Rationale
Config-Driven Dispatch Engine	Build	Core differentiator; no vendor product encodes ForkRoute-specific marketplace dynamics and rider-matching logic (ADR-001).
Market Config Store & Validation Harness	Build	Tightly coupled to the Dispatch Engine's internal contract; not a generic product.
Route Optimization / ETA Engine	Buy	Commodity capability with mature vendor options; building in-house consumes scarce algorithms capacity better spent on dispatch itself (Principle 7).
Dynamic Surge Pricing Calculator	Build	Direct expression of ForkRoute's marketplace economics and competitive positioning; not a candidate for a generic vendor product (ADR-004).
Multi-Tenant Restaurant Integration Platform	Build (on bought API gateway)	Tenant isolation logic is ForkRoute-specific; underlying gateway/rate-limiting infrastructure is bought (ADR-003).
Payments Event Log & Continuous Reconciliation	Build (on bought event streaming)	Reconciliation business logic (commission rules, disputes) is ForkRoute-specific; the durable log infra itself is bought/managed (ADR-002).
Payout Orchestrator	Hybrid	Orchestration logic built in-house; actual funds movement delegated to LedgerBridge, the selected payments platform.
Jurisdiction-Aware Data Residency Layer	Hybrid	Zone architecture and enforcement policy designed in-house; regional infrastructure provisioning uses standard cloud-provider residency features (ADR-005).
Multi-Currency Financial Primitives Library	Build	Must be embedded consistently across every ForkRoute-owned financial code path; no vendor substitute for internal discipline.
Architecture Governance Tooling	Buy	Not a differentiating capability; standard exception-register/decision-log/contract tooling is entirely adequate (Principle 7).

☐ Buy/Configure
 ☒ Build (new)
 ☐ Build (ForkRoute-specific)
 ☐ Hybrid buy+build

Source: 05-phase-e-opportunities-and-solutions/solution-building-blocks.md

Diagram 14 — Solution Building Blocks, Buy / Build / Hybrid

7.3 Vendor Evaluation

Two buy/hybrid decisions required formal vendor evaluation against seven weighted criteria: multi-market/multi-currency support (20%), integration effort with the event-driven backbone (15%), real-time/low-latency capability (15%), data residency and compliance posture (15%), 3-year total cost of ownership (15%), vendor stability and support quality (10%), and extensibility/API openness (10%). For the Route Optimization/ETA Engine, RouteVantage and PathForge Logistics landed on an exact tied weighted score of 3.85 — the honest result of two genuinely different strength profiles, not a close call rounded away. PathForge led on multi-market breadth and data residency; RouteVantage led on real-time capability and integration effort, the two criteria most directly load-bearing for the surge-pricing feedback loop.

RouteVantage won the tiebreak because PathForge's integration model assumes synchronous polling rather than webhook/event delivery, which would have required a translation layer sitting awkwardly against Principle 6's contract-versioning standard — an ongoing integration tax, not a one-time cost. NaviCore Fleet, the third candidate, scored 3.70, losing primarily on multi-market currency and locale support, a harder gap to work around than an integration adapter given three of the four target markets have distinct currency and formatting requirements out of the gate.

For the Payments/Reconciliation Platform beneath the in-house Payout Orchestrator, LedgerBridge won with a weighted score of 4.10, the clear leader on the two criteria weighted heaviest for this program's specific problem: multi-market support and real-time capability, both directly load-bearing for the sub-30-minute payout target and the surge-split-to-payout requirement. SettleWorks (3.90) offered the easiest integration of the four — a native event-webhook model closely matching the reference architecture — but its weaker residency posture, with data processing centralized in a single region regardless of transaction origin, conflicted directly with Principle 4 and would have required a costly custom data-routing layer to compensate. ClearPay Rails (3.55) had the strongest residency and real-time scores, but its 3-year TCO is an illustrative ~40% higher than LedgerBridge's and its extensibility score is the weakest of the four — its API does not expose enough control over commission-split logic to implement ForkRoute's specific surge-split rules without a support-ticket-driven customization cycle, an unacceptable dependency for a Tier 1 capability. Meridian Settlement (3.75) had the best 3-year TCO by a wide margin, but its real-time capability is the weakest of the four — a batch-oriented core with a "near-real-time" add-on still measuring latency in tens of minutes, not seconds — in effect a lower-cost version of the batch model this program exists to replace, and selecting it would have undermined the program's own stated objective for cost reasons alone. Both recommendations were reviewed and approved by the full ARB given their budget impact exceeds the \$150,000 threshold, with Finance consulted specifically on the TCO scoring.

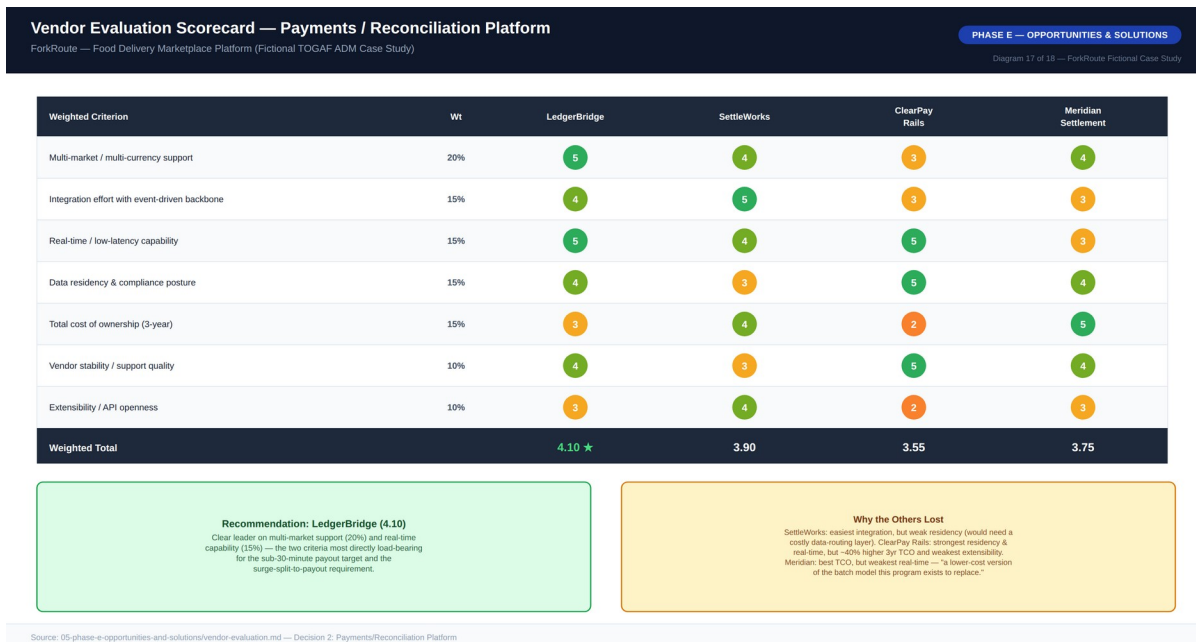


Diagram 17 — Vendor Evaluation Scorecard, Payments/Reconciliation Platform

8. Phase F — Migration Planning

8.1 Migration Roadmap

The roadmap runs across a Foundation period and four waves over 18 months, sequenced by the dependency logic established in the gap analysis: foundational, high-dependency gaps — data residency, then the payments backbone, then the dispatch engine — are built first against the home market, with parallel work on restaurant integration isolation, and expansion-market rollouts begin only once the home-market platform has proven itself in production, not on paper. Foundation (Months 1–5) builds the three foundational layers against home-market production traffic without cutting over; data residency zone architecture is sequenced first and given the longest lead time relative to its complexity score specifically because every subsequent block depends on it, while restaurant integration isolation runs as a genuinely parallel workstream with a separate delivery team, per the gap analysis's determination that it has no hard dependency on the payments/dispatch work.

Wave 1 (Months 6–8) introduces Transition State Alpha: dispatch and payments run dual-run — old and new systems operating in parallel with output reconciled against each other — before the home market fully cuts over. This is the highest-risk wave in the program, and it is deliberately run against the home market, where ForkRoute has operational maturity and brand trust to absorb any turbulence, rather than against a new market with neither. Wave 2 (Months 9–11) launches Market A directly on the re-architected platform — no dual-run needed, since there is no legacy Market A system to run in parallel with; this is the first live test of the "new market as configuration" thesis the business case rests on, and its actual lead time against the 3-week target is the single most-watched metric by the Executive Steering Committee. Wave 3 (Months 12–14) launches Markets B and C in parallel, testing whether the platform genuinely supports concurrent market onboarding without config-authoring or vendor-integration teams becoming a bottleneck — a deliberate stress test before committing to Wave 4. Wave 4 (Months 15–18) launches Market D, then completes formal program closeout: governance responsibility for the platform transitions from the program's dedicated ARB structure to ForkRoute's standing engineering organization.

Each wave transition is a go/no-go checkpoint owned by the Executive Steering Committee, informed by the Chief Architect's technical readiness certification. A wave is not permitted to begin until the prior wave's success metrics — dispatch lead time, payout latency, incident containment — have been validated in production, not merely in staging. This is a deliberate constraint accepted at the cost of some schedule risk, in exchange for not compounding an unvalidated architecture decision across multiple markets simultaneously.

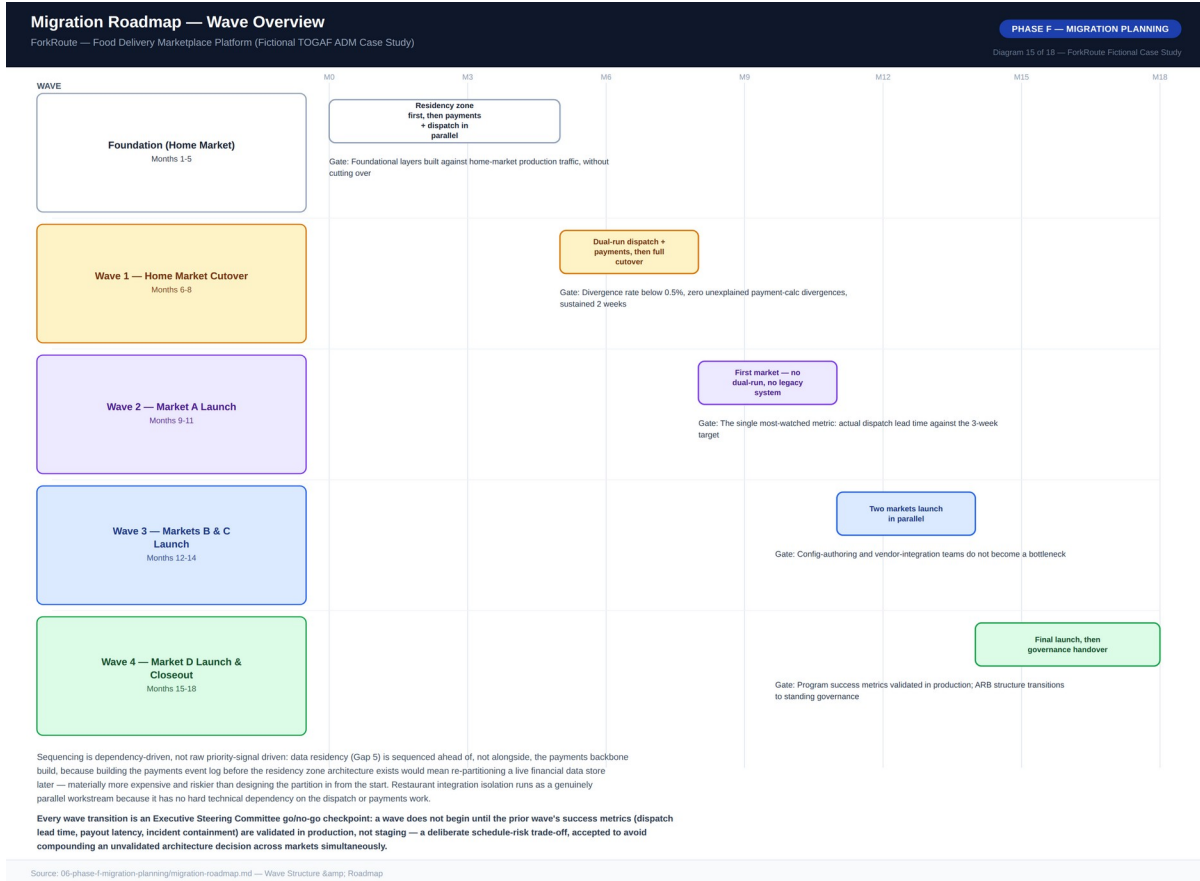


Diagram 15 — Migration Roadmap, Wave Overview

8.2 Transition Architecture — Home Market Dual-Run

A migration plan that only describes as-is and to-be leaves the riskiest period of the program — where both systems are partially true at once — undocumented. Transition State Alpha (Wave 1, home market only) names and describes exactly what is true, and what is not yet true, during dual-run. The config-driven dispatch engine and event-driven payments backbone are both live in production, receiving a shadow copy of real order traffic, but the legacy city-coded dispatch service and nightly batch reconciliation remain the systems of record for actual rider offers and actual payout instructions. Every order is processed by both paths; a comparison job — distinct from, and simpler than, the eventual Continuous Reconciliation Service — compares the two systems' dispatch decisions and payment calculations and flags divergence for engineering review. Customers, riders, and restaurants experience no behavior change during this state: they are served entirely by the legacy path, and the new path's output is observed, not acted upon.

This state is necessary because the event-driven payments backbone is a Tier 1, financial-impact capability; cutting it over without a validation period against known-correct legacy output would mean discovering a reconciliation bug in production with real money at stake and no baseline to compare against. Dual-run converts "trust the new system because we tested it" into "trust the new system because we watched it agree with the old one across weeks of real traffic." Exit criteria are binding, not a target to aim for loosely: a divergence rate between legacy and new-path dispatch decisions below 0.5%, and zero unexplained payment-calculation divergences, sustained for a minimum two-week window, before proceeding to full cutover — a threshold set by the VP Engineering, Payments, in consultation with the ARB. At full cutover, the

new platform becomes the system of record and the legacy dispatch service and batch reconciliation job are decommissioned for the home market, not merely deprioritized.

Two further transition states run alongside Alpha. Transition State Bravo (concurrent with Alpha, since restaurant integration isolation was sequenced as a parallel workstream) is a restaurant integration platform coexistence: new partners onboard directly onto the tenant-isolated platform, existing partners migrate off the legacy shared tier in scheduled batches grouped by traffic profile — large chains first, in small, closely monitored batches, then smaller independents in larger batches once the pattern is validated on the higher-risk chains — because migrating all 15,000 existing partners in a single cutover would concentrate migration risk into one event with no ability to isolate or roll back a single problematic partner's migration. Transition State Charlie (between Wave 1 and Wave 2) is multi-currency partial coverage: the Multi-Currency Financial Primitives Library is fully implemented and enforced in the Payments Event Log and Payout Orchestrator, but only the home market's currency is actually in live use, with multi-currency support validated through synthetic test transactions and a controlled internal pilot rather than live customer traffic — a deliberate "build the capability before you need it, but validate it before you need it too" state, so a foundational financial-correctness question is not sitting on the critical path of Market A's actual launch.

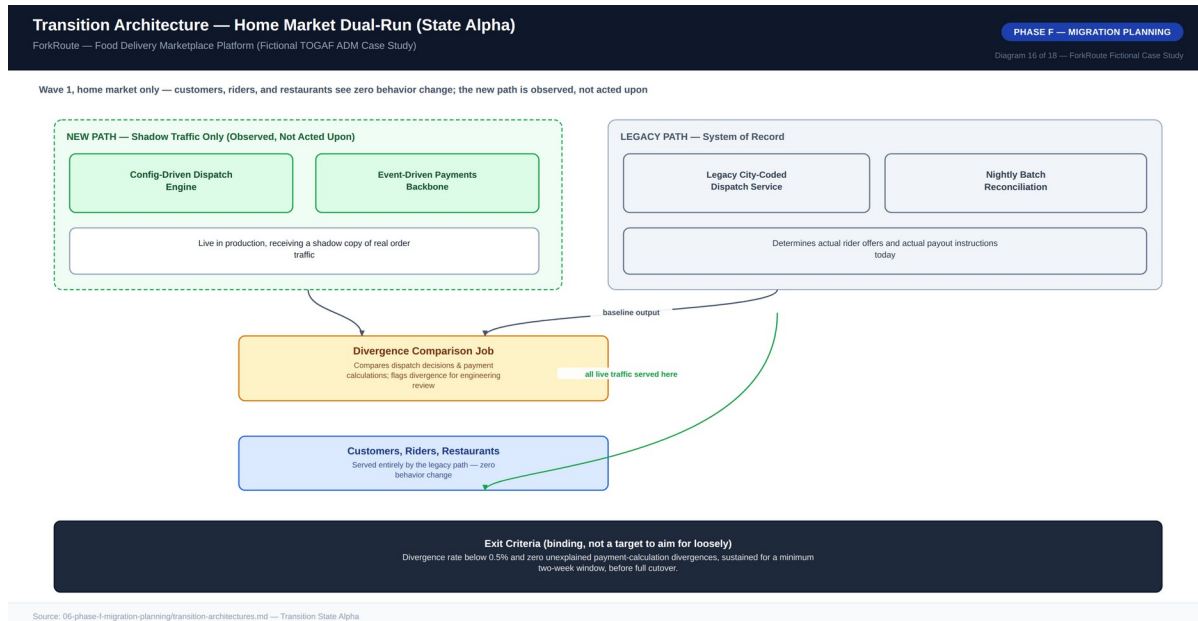


Diagram 16 — Transition Architecture, Home Market Dual-Run (State Alpha)

9. Phase G — Implementation Governance

Phase G governs how the target architecture defined in Phases B through E is actually built, ensuring delivery teams stay within the approved architecture without turning the ARB into a bottleneck for routine engineering. Architecture compliance is checked at three points in each delivery team's lifecycle, not only at the start. Design review, before implementation begins, checks the proposed design against the architecture principles and technology standards, with alternatives considered documented, and is performed at the weekly design review. Contract sign-off, before a delivery team is resourced, requires an architecture contract signed by the Chief Architect and the delivery team's engineering lead. Pre-production readiness review, before a building block goes live — including before each wave's go/no-go — checks that contract terms are actually met and that deviations are logged as exceptions with sign-off, not silently absorbed, performed by the ARB. A fourth, lighter-weight checkpoint, a quarterly compliance audit, spot-checks live building blocks against their original contracts even after go-live, specifically to catch drift that accumulates gradually — a config bundle that grew an undocumented special-case rule, for instance — rather than arriving as a single reviewable change.

9.1 Non-Compliance Handling

Non-compliance is treated as a spectrum, not a binary pass/fail, and the response is proportionate. Minor, correctable drift — a missing correlation-ID field in a new event type, for example — is flagged at the next design review or pre-production readiness review, corrected before go-live, with no escalation needed. Material deviation from a signed contract without an approved exception escalates to the full ARB; the delivery team's engineering lead and the Chief Architect jointly present the situation, and the ARB's options are to retroactively approve an exception if the deviation is sound but wasn't pre-cleared, require remediation before proceeding further, or, in the most serious case, pause the workstream. Repeated non-compliance from the same delivery team is treated as a signal that the contract itself, the team's capacity, or the review process is miscalibrated, and is raised as a standing agenda item at the bi-weekly program governance review rather than re-litigated incident by incident. Non-compliance is never handled by quietly updating documentation to match what was actually built after the fact — that would defeat the entire purpose of Principle 8. Every deviation, whether it turns out to be a good idea or not, is a first-class recorded event.

9.2 Architecture Contracts

An architecture contract is the binding agreement between the EA function and a delivery team, defining exactly what the team is accountable for building, against which principles and standards, with what acceptance criteria. It contains a scope statement referencing the specific solution building block(s) covered; the applicable principles and technology standards the team must design against; the interfaces and contracts owed to other teams, per Principle 6; non-functional requirements — availability tier, latency targets, data residency constraints — per the reference architecture's criticality tiering; acceptance criteria, the specific, testable conditions under which the ARB will sign off at the pre-production readiness review; an exception log reference for any pre-approved deviations; and signatories, the Chief Architect and delivery-team engineering lead, both required before resourcing begins — this ordering is deliberate, since signing before resourcing means the team is staffed against an agreed, reviewed scope, not a scope renegotiated after people are already committed to a sprint plan.

The worked example is the Config-Driven Dispatch Engine contract, held by Platform Engineering's Dispatch Squad, citing Principle 1 (Configuration Over Code), Principle 5 (Every New Market Is a Configuration), Principle 6 (Interfaces Are Contracts), and Principle 10 (Resilience Scoped to Business Impact). It requires the engine to publish DispatchOffered, DispatchAccepted, DispatchDeclined, and SurgeApplied events to the payments event backbone, versioned per the schema registry; to expose a synchronous POST /dispatch/offer API to the Order Service with a p99 latency target under 800ms, within the program's 2-second end-to-end dispatch SLA, leaving headroom for upstream/downstream legs; and to consume MarketConfigBundle versions from the Market Config Store, rejecting and alerting on any bundle that fails schema validation rather than falling back silently. Non-functional requirements set the engine to Tier 1 (99.95% availability target) and require every dispatch decision to log which config bundle version was active, satisfying the audit traceability requirement from Security & Compliance. Acceptance criteria checked at the pre-production readiness review include: zero city-keyed conditional branches remaining in the dispatch codebase, verified by code review plus a test that runs the same binary against two different config bundles and confirms divergent behavior with no code change; an operational dry-run simulation harness exercised against at least one real historical dataset, reviewed by the relevant Regional Expansion Lead; the p99 latency target met under a load test simulating projected peak volume for the largest target market; and a divergence rate against the legacy dispatch service, measured during Transition State Alpha, below the 0.5% exit threshold for a sustained two-week window. No exceptions were logged against this contract at signing; one exception was later logged during delivery for a temporary synchronous payout-integration fallback unrelated to this block's own scope, cross-referenced in the exception register.

10. Phase H — Change Management

An architecture that is technically correct but organizationally unadopted delivers none of the business case's projected value — the dispatch-lead-time, incident-avoidance, and reconciliation-labor savings depend entirely on operations managers, regional leads, and partner-facing teams actually using the new capabilities as designed. This plan treats organizational adoption as a first-class deliverable with the same governance rigor as the technical architecture, and treats its single highest-severity change honestly rather than reframing it as a pure upgrade: City Operations Managers losing manual surge-pricing control is a genuine reduction in a role's discretionary authority, not merely a new tool to learn, and operations managers are being asked to trust an automated system with a decision they used to make by judgment. The communication and training approach is built around earning that trust, not assuming it.

Affected Group	Nature of the Change	Impact Severity
City Operations Managers	Lose direct manual control over surge pricing; shift from an operator role to a config-authoring and monitoring role	High — an authority change, not just a tooling change
Restaurant Partnerships Team	Shift from uniform hands-on onboarding to a triage model (self-service standard cases, hands-on complex ones)	Medium — workload composition changes
Solutions Engineering	Time reallocated away from routine onboarding toward complex-partner integration work	Medium — role scope narrows and deepens simultaneously
Platform & Payments Engineering	New event-sourcing and config-governance disciplines required; new operational runbooks for the event backbone	High — a genuinely new skill set
Customer Support Leadership	New dispute-handling workflow using the payments event log's audit trail instead of next-day batch reports	Medium — strictly more capable than what it replaces
Regional Expansion Leads	New responsibility for config bundle authoring and validation sign-off per market	High — new work these roles did not previously do
Finance	New real-time payout reconciliation view replaces monthly close-cycle-aligned batch reports	Medium — improves visibility but adjusts close-process assumptions

Training is aligned to the wave structure. City Operations Managers get hands-on workshops using the config dry-run simulation harness — the same tool used in the architecture contract's acceptance criteria — against their own city's historical data, so they can see the automated engine's decisions against known outcomes

before it goes live, completed ahead of their city's Transition State Alpha exit. Regional Expansion Leads complete a structured config-authoring certification, a supervised config bundle authoring and validation exercise, before being authorized to sign off a market's live bundle independently, ahead of each market's Wave 2/3/4 config-authoring phase. Platform & Payments Engineering get an internal event-sourcing and config-governance onboarding curriculum plus paired on-call rotations during Transition State Alpha, building operational familiarity under real but shadow-mode load. Restaurant Partnerships and Solutions Engineering get a workshop on the new self-service onboarding flow and triage criteria ahead of Transition State Bravo's partner migration batches. And Customer Support Leadership gets a walkthrough of the new payments audit-trail tooling, side-by-side against the old batch-report workflow, ahead of Wave 1 home-market cutover.

Program kickoff communication to all affected groups explains the business driver and, deliberately, what is not changing — customer, rider, and restaurant-facing app interfaces are unchanged — to prevent rumor-driven overestimation of the change's scope. The Executive Steering Committee receives bi-weekly written status reports tied to the program governance review; City Operations Managers get monthly updates during the Foundation wave and weekly updates during their city's transition window; Restaurant Partnerships and Solutions Engineering get monthly updates with a dedicated session ahead of each partner migration batch; engineering delivery teams stay current via published ARB minutes; and external restaurant partners get advance notice ahead of their own migration batch with a defined support contact.

Adoption is measured alongside technical migration metrics at the bi-weekly program governance review, on the basis that a technically successful migration with poor human adoption is not, in fact, a successful program outcome. Targets: 100% of City Operations Managers complete a supervised dry-run exercise before their city's Transition State Alpha exit; 100% of Regional Expansion Leads complete config-authoring certification before their market's Wave 2/3/4 config-authoring phase begins; a declining trend in support tickets referencing surge-pricing confusion over the first 60 days post-cutover, benchmarked against first-week volume; at least 70% of new restaurant partner onboardings completed via self-service, with no solutions-engineer escalation, within 90 days of Transition State Bravo's completion; and an average pre/post training confidence-survey improvement of at least 1.5 points (1–5 scale) among Platform and Payments Engineering.

11. Architecture Decision Records

Five ADRs anchor the architecturally significant decisions in this program. Each follows the standard Status / Context / Decision / Alternatives Considered / Consequences / Governance format and is cross-referenced from the phase documents and architecture contracts that depend on it.

ADR-001: Config-Driven Dispatch Engine

Replace the city-coded dispatch service with a single, config-driven engine whose market-specific behavior is entirely determined by a versioned, schema-validated configuration bundle loaded at runtime. A single global dispatch service with no per-market variation was rejected because ForkRoute's markets genuinely differ in ways that matter to dispatch quality — delivery density, trip distance, regulatory constraints on driver working patterns — and a one-size-fits-all rule set would degrade quality in at least some markets. A per-market dispatch service with shared libraries was seriously considered — it offers stronger per-market blast-radius isolation, since a bug in one market's service cannot directly affect another's runtime — but was rejected because it formalizes the fork rather than eliminating it and still requires a code deployment to change market behavior; the ARB judged the ongoing maintenance burden of N deployable services to outweigh the isolation benefit. Accepted trade-offs: the runtime rules-interpretation layer adds an estimated 8–12ms of latency per dispatch decision versus compiled market-specific code, judged acceptable against the program's 2-second end-to-end SLA (the architecture contract's p99 target of 800ms retains ample headroom); configuration becomes a new, first-class failure mode without the safety net of code review and CI, mitigated but not eliminated by treating config bundles as governed master data; and engineers must design market-parameterizable logic from the start, increasing upfront design time for the home-market migration in exchange for near-zero incremental engineering cost for markets 2 through 5. Approved by the full ARB given cross-cutting impact on Platform Engineering, the Regional Expansion Leads, and the program's headline success metric; implements Principle 1 and Principle 5.

ADR-002: Event-Driven Payments Reconciliation

Replace batch reconciliation with an event-sourced payments architecture: every financial state change is written as an immutable event to an append-only, replayable log, the authoritative system of record, with reconciliation as a continuous process consuming the log rather than a nightly job reading transactional snapshots. Keeping nightly batch but running it more frequently (e.g., hourly) was seriously considered as a lower-risk, lower-cost incremental improvement, and rejected specifically because it would not meet the sub-30-minute payout target and does not solve the harder problem of correctly calculating and threading a dynamic surge split through to payout, which requires knowing the pricing decision at the moment it was made, not reconstructing it from a batch snapshot hours later. A synchronous, strongly-consistent ledger with direct calls from every consumer to the payments service was attractive for its simplicity but rejected because it creates a shared-fate dependency — a slow or down reconciliation service should never be able to affect live payment authorization, which a synchronous coupled design makes possible. Accepted trade-offs: event-sourcing discipline (idempotency keys, schema versioning, replay tooling, dead-letter handling) is new, materially higher-bar operational surface area for teams used to transactional-table-plus-batch patterns, with direct financial and regulatory consequences if done wrong; eventual consistency is now a property every downstream consumer must reason about, mitigated by differentiated criticality tiering; and the nightly-batch general ledger integration is not being replaced in this program, so the event log's output must be adapted to feed that legacy integration — accepted technical debt pending a separate future finance-systems

initiative. Approved by the full ARB with VP Engineering, Payments as Responsible and Chief Architect as Accountable; implements Principle 3.

ADR-003: Multi-Tenant Restaurant Integration Platform

Re-architect the restaurant integration platform to be explicitly multi-tenant: each partner, or partner group for chains, is provisioned with isolated rate limits, dedicated processing lanes, and independent circuit breakers, so one partner's anomaly cannot degrade the platform for others. Global rate limiting with a higher ceiling sized for the expansion partner count was considered as a low-cost interim measure and rejected as insufficient for the target state — it protects the platform in aggregate but does nothing to stop one noisy partner from starving others within that ceiling. Fully separate integration infrastructure per partner was rejected as operationally unworkable at ForkRoute's scale — provisioning and maintaining 15,000+ (or even a meaningful subset of large-chain) fully separate deployments would consume far more operational overhead than the isolation benefit justifies, and most independent restaurants generate traffic volumes low enough that dedicated infrastructure would be significant over-provisioning. Accepted trade-offs: per-tenant quota management, monitoring, and circuit-breaker configuration is ongoing operational surface area, not a one-time setup cost; existing partners must migrate from the shared tier to the isolated tier in batches, extending the migration timeline versus a theoretical single cutover in exchange for containing migration risk; and very low-traffic partners receive isolation infrastructure sized for a risk profile they individually pose little of, accepted as the cost of a uniform, predictable isolation model over a partner-tier-dependent one. Approved by the ARB with VP Engineering, Platform Responsible and Restaurant Partnerships consulted; implements Principle 2.

ADR-004: Surge Pricing Architecture

Implement surge pricing as an automated Dynamic Surge Pricing Calculator, part of the dispatch engine domain, continuously evaluating live driver-supply and demand signals within each market's configured bounds (minimum/maximum multiplier, evaluation frequency, triggering thresholds, all expressed in the market's config bundle) and emitting a SurgeApplied event as a first-class, auditable part of the order's event trail feeding directly into the Payments Event Log. Retaining the manual toggle but making it more granular (neighborhood-level rather than citywide) was considered as a lower-risk incremental step and rejected because it does not solve the core problem — it remains a human-judgment-driven decision with no live data feed and no auditable, structured signal for the payments backbone, effectively deferring the real decision rather than making it. A fully external, vendor-provided dynamic pricing engine was rejected under the same reasoning as the solution-building-blocks build-vs-buy analysis: surge thresholds, caps, and fairness constraints are strategic business logic specific to ForkRoute's competitive position, not a commodity capability. Accepted trade-offs: City Operations Managers lose direct manual control over surge — a genuine authority change carrying real adoption risk, addressed as a first-class change-management risk rather than a side effect to be absorbed silently; an automated pricing engine raises new governance questions (fairness bounds, regulatory scrutiny of algorithmic pricing) that a manual toggle did not surface as sharply, with the config bundle's explicit multiplier bounds as the ARB's primary control and an area flagged for continued attention as expansion-market regulatory posture becomes clearer; and building this in-house means ForkRoute bears full engineering cost and maintenance for the pricing algorithm, accepted as a deliberate differentiation investment. Approved by the ARB with VP Engineering, Platform Responsible for dispatch-side integration and VP Engineering, Payments Responsible for payout-split integration, Chief Architect Accountable; extends Principle 1 and Principle 3.

ADR-005: Market-Expansion Data Residency Strategy

Treat data residency zone as a first-class partitioning dimension: each target market, or group of markets sharing compatible legal frameworks, is assigned to a residency zone at design time, with PII and transactional payment data stored and processed within that zone by infrastructure-level enforcement, not application-level filtering alone. Cross-market aggregate reporting runs on de-identified or aggregated exports pushed to a central analytics zone rather than live cross-region queries. A single global data plane with application-level residency filtering was the lowest-cost option and was seriously considered for that reason, but rejected because application-level filtering is a defense a single application bug can bypass — a query that forgets to apply the jurisdiction filter would leak cross-zone data with no infrastructure-level backstop, a risk the ARB and Head of Security & Compliance judged unacceptable given real regulatory consequences in the affected markets. A live, federated cross-region query layer for both operational and analytics use cases was rejected primarily on residency-compliance grounds — it makes it easier, not harder, to accidentally construct a cross-boundary query — and secondarily on latency and operational-complexity grounds. Accepted trade-offs: cross-market aggregate reporting becomes dependent on a daily export-and-aggregate pipeline rather than a live query, adding latency (same-day, not real-time) to global reporting, communicated explicitly to Finance and executive stakeholders rather than discovered as a surprise after launch; every foundational building block has a hard dependency on the residency zone architecture being in place first, concentrating schedule risk at the very start of the program; and operating multiple isolated infrastructure zones is a genuinely higher operational cost than a single global deployment, reflected in the business case's cloud infrastructure delta. Approved by the full ARB; Head of Data Responsible for technical implementation, Regional Expansion Leads Responsible for confirming local regulatory interpretation per market, Chief Architect Accountable overall; implements Principle 4.

Appendix A — Diagram Catalog Index

The full reference diagram catalog for this program comprises 18 diagrams spanning the Preliminary Phase through Phase F, delivered as high-resolution PNG files alongside this document.

No.	Diagram Title	ADM Phase
01	Architecture Vision One-Pager	Phase A
02	Stakeholder Power / Interest Map	Phase A
03	Architecture Governance Organization	Preliminary
04	Business Capability Map, Maturity Heat Map	Phase B
05	Business Process Flow, As-Is (Order Dispatch & Driver Allocation)	Phase B
06	Business Process Flow, To-Be (Order Dispatch & Driver Allocation)	Phase B
07	Business Footprint Diagram	Phase B
08	Application Communication Diagram, As-Is	Phase C
09	Application Communication Diagram, To-Be	Phase C
10	Data Flow Diagram, To-Be (Jurisdiction-Aware Residency)	Phase C
11	Data Ownership / Domain Diagram	Phase C
12	Reference Architecture Pattern	Phase D
13	Technology Standards Stack	Phase D
14	Solution Building Blocks, Buy / Build / Hybrid	Phase E
15	Migration Roadmap, Wave Overview	Phase F
16	Transition Architecture, Home Market Dual-Run (State Alpha)	Phase F
17	Vendor Evaluation Scorecard, Payments/Reconciliation Platform	Phase E
18	Business Case Financial Summary	Phase A

Appendix B — Disclaimer

This entire document — ForkRoute, all named individuals and roles, all vendor names (RouteVantage, PathForge Logistics, NaviCore Fleet, LedgerBridge, SettleWorks, ClearPay Rails, Meridian Settlement), and every dollar figure, metric, and date — is a fictional case study constructed for architecture-practice demonstration purposes. Figures are designed to be internally consistent and arithmetically sound so that the reasoning pattern is transparent, not to represent any real food-delivery company's actual data. No inference should be drawn about any real company, product, or individual.